2026 edition

Deep Learning for Music Analysis and Generation

Vocoders

(mel → audio)



Yi-Hsuan Yang Ph.D.
yhyangtw@ntu.edu.tw

Objectives

- Traditional DSP-based **source-filter** model for audio
- **Upsampling** layers in DL models for audio
- The design of **GANs** for audio

- It's a type of *conditional* audio **generation** task (with *strong condition*)
  - **"mel → audio"**

Reference 1: ICASSP 2022 Short Course

https://www.slideshare.net/jyamagis/advancements-in-neural-vocoders

https://colab.research.google.com/drive/1EO-ggi1U9f2zXwTiqg7AEljVx11JKta7



Tutorial on neural vocoder.

By Xin Wang, National Institute of Informatics, 2022. See more in LICENSE Section.

This was used for ICASSP 2022 short course **Inclusive Neural Speech Synthesis**, **LECTURE 1, vocoder**.

Full Table of contents:

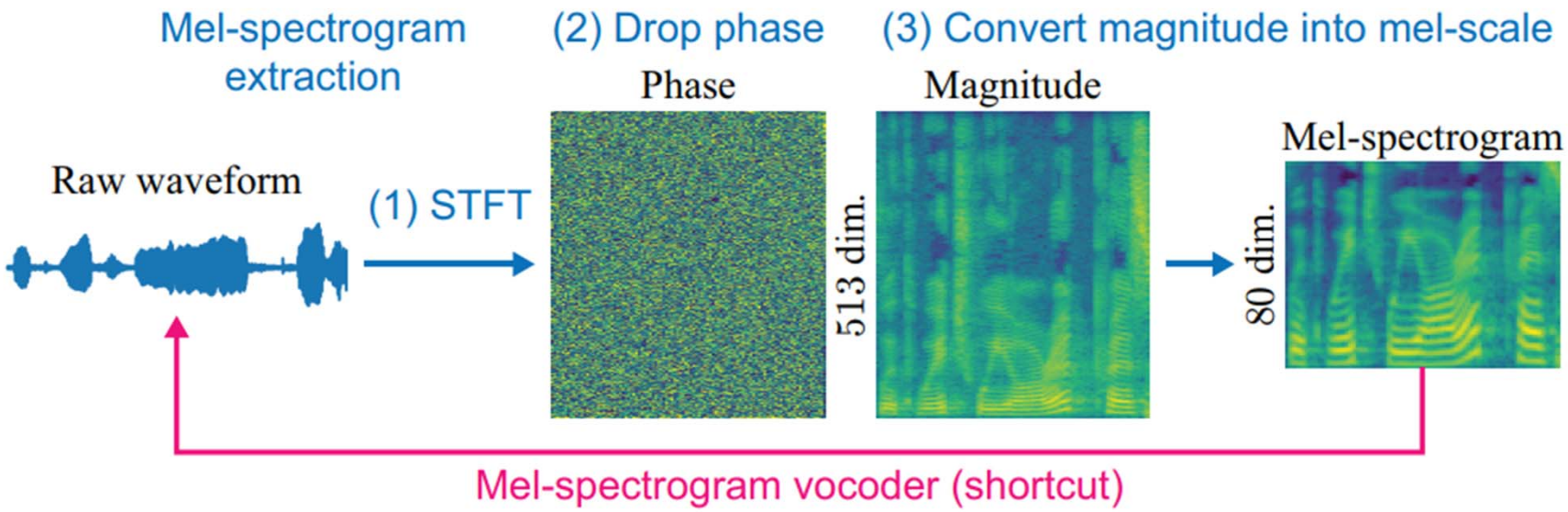
- Introduction & basics:
  - chapter_1_introduction.ipynb: entry point, and Python/Pytorch conventions used in this hands-on session;
  - chapter_2_DSP_tools_Python.ipynb: selected DSP tools for speech processing;
  - chapter_3_DSP_tools_in_DNN_Pytorch.ipynb: selected DSP tools implemented as layers in neural networks;
- DSP-based Vocoder:
  - chapter_4_DSP-based_Vocoder: traditional DSP-based vocoder included in SPTK toolkit;
- Neural vocoders:
  - chapter_5_DSP+DNN_NSF.ipynb: a simple neural vocoder without autoregressive, GAN, or flow;
  - chapter_6_AR_WaveNet.ipynb: the autogressive WaveNet;
  - chapter_7_AR_iLPCNet.ipynb: the autogressive iLPCNet;
  - chapter_8_Flow_WaveGlow.ipynb: the flow-based WaveGlow model;
  - chapter_9_GAN_HiFiGAN_NSFw/GAN.ipynb: the simple but high-quality HiFiGAN vocoder. We also show the NSF combi

Outline

- **General idea**
- DL components: upsampling layers and GAN
- GAN-based Mel-vocoders

Mel-Vocoder

- **Mel-vocoder**: Mel-spectrogram → audio waveforms



Ref: Figure from https://arxiv.org/pdf/2203.02395.pdf

Mel-Vocoder

- **Mel-vocoder**: Mel-spectrogram → audio waveforms
- For not only speech but **any sounds**
- We can divide an audio synthesis model into two steps:
  - The **generator** generates acoustic features, or the Mel-spectrogram
  - The **vocoder** converts Mel-spectrogram into waveforms
- Why Mel-vocoder becomes popular in the DL era
  - More information than F0 + spectral envelop + aperiodicity
  - Modern computing devices can deal with acoustic features with a larger dimension

Mel-Vocoder: Approaches

- Pure **signal processing** techniques
  - Griffin & Lim algorithm
- **Autoregressive** neural networks
  - WaveNet, SampleRNN, WaveRNN
  - "Inference with these models is inherently **slow** and inefficient because audio samples must be **generated sequentially**"
- **Non-autoregressive** neural networks
  - "Orders of magnitudes faster than their auto-regressive counterparts because they are highly parallelizable and can fully exploit modern deep learning hardware"

Ref: Quotes from the MelGAN paper (https://arxiv.org/pdf/1910.06711.pdf)

Griffin & Lim Algorithm

- Initially for magnitude spectrogram → audio waveforms
- Iterative optimization
- No training is needed
- Robotic artifacts

算法步骤：

- 随机初始化一个相位谱
- 用这个相位谱与已知的幅度谱（来自MEL谱）经过ISTFT（逆傅里叶变换）合成新的语音波形
- 用合成语音做STFT， 得到新的幅度谱和新的相位谱
- 丢弃新的幅度谱，用已知幅度谱与新的相位谱合成新的语音
- 重复2,3,4多次，直至合成的语音达到满意的效果或者迭代次数达到设定的上限

https://blog.csdn.net/CSDN_71560364126/article/details/103968034

Ref: Griffin & Lim, “Signal estimation from modified short-time Fourier transform,” TASLP 1984.

Griffin & Lim Algorithm

https://speechprocessingbook.aalto.fi/Modelling/griffinlim.html

```
# example signal
original_waveform = np.sin(np.linspace(0,221,400))
_,_,original_spectrogram = scipy.signal.stft(original_waveform,nperseg=100, return_onesided=True)
frames = original_spectrogram.shape[1]

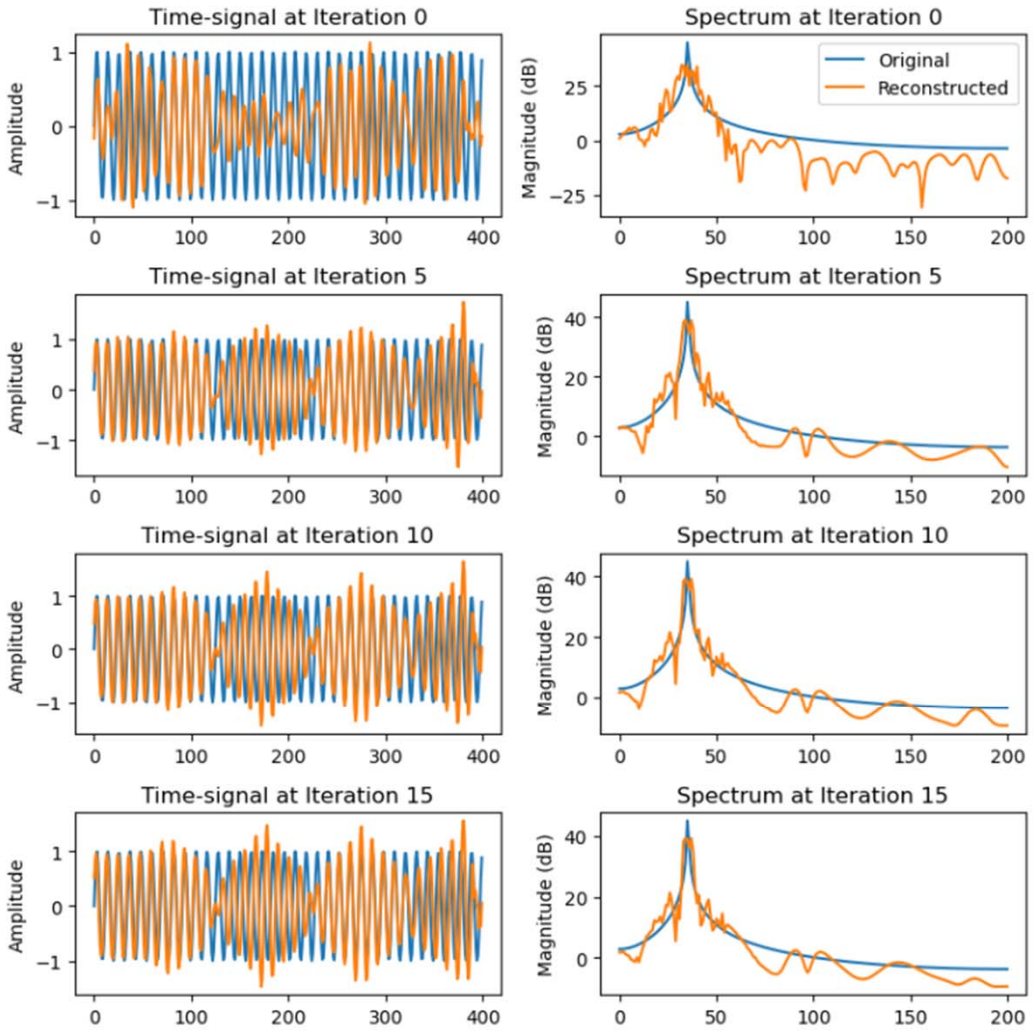
# initialize target magnitude spectrogram
magnitude_spectrogram = np.abs(original_spectrogram)

# initial guess of complex spectrogram = random phase
spectrogram = magnitude_spectrogram*np.exp(1j*2*np.pi*np.random.rand(51,frames))
```

```
plt.figure(figsize=(8,8))
N = 4
K = 5
for h in range(N*K - K+1):
    _,waveform = scipy.signal.istft(spectrogram, nperseg=100, input_onesided =True, boundary=True)
    waveform = np.real(waveform)
    _,_,spectrogram = scipy.signal.stft(waveform,    nperseg=100, return_onesided=True)
    spectrogram = np.exp(1j*np.angle(spectrogram))*magnitude_spectrogram

    if h % K == 0:
        k = h//K
        plt.subplot(N,2,2*k+1)
        plt.plot(original_waveform)
        plt.plot(waveform)
        plt.title(f"Time-signal at Iteration {h}")
        plt.ylabel('Amplitude')

        plt.subplot(N,2,2*k+2)
        plt.plot(20*np.log10(np.abs(scipy.fft.rfft(original_waveform))),label='Original')
        plt.plot(20*np.log10(np.abs(scipy.fft.rfft(waveform))),label='Reconstructed')
        plt.title(f"Spectrum at Iteration {h}")
        plt.ylabel('Magnitude (dB)')
        if h==0: plt.legend()
```



Griffin & Lim-based Mel-Vocoder via LibROSA

https://librosa.org/doc/main/generated/librosa.feature.inverse.mel_to_audio.html#librosa.feature.inverse.mel_to_audio

librosa.feature.inverse.mel_to_audio(*M, *, sr=22050, n_fft=2048, hop_length=None, win_length=None, window='hann', center=True, pad_mode='constant', power=2.0, n_iter=32, length=None, dtype=<class 'numpy.float32'>, **kwargs*) [source]

Invert a mel power spectrogram to audio using Griffin-Lim.

This is primarily a convenience wrapper for:

```
>>> S = librosa.feature.inverse.mel_to_stft(M)
>>> y = librosa.griffinlim(S)
```

librosa.util.nnls(*A, B, **kwargs*) [source]

Non-negative least squares.

Given two matrices A and B, find a non-negative matrix X that minimizes the sum squared error:

```
err(X) = sum_i,j ((AX)[i,j] - B[i, j])^2
```

- **mel-to-stft**: approximate STFT magnitude from a Mel power spectrogram using a non-negative least squares (NNLS) algorithm

Non-autoregressive Neural Vocoders

- Via **knowledge-distillation** of autoregressive models
  - Parallel WaveNet (Oord et al., 2017) and Clarinet (Ping et al., 2018)
- Via **flow-based** models
  - WaveGlow (Prenger et al., 2019)
- Via **GAN**-based models
  - Reconstruction loss + adversarial loss
  - SOTA

Ref: Quotes from the MelGAN paper (https://arxiv.org/pdf/1910.06711.pdf)

Why GAN? (Or, Why Reconstruction Loss Alone is Not Enough)

This new methodology … has allowed for waveforms to be generated … at … a higher real-time speed than in models with autoregressive constraints

MelGAN is substantially faster than other mel-spectrogram inversion alternatives

Multi-band MelGAN runs at more than 10x faster than real-time on CPU.

- Not only about fast inference speed
- Also on combating with the over-smoothing

*(Conventional) models are limited due to the point-wise regression in the time domain, which results in **smoothing and loss of high frequencies**.*

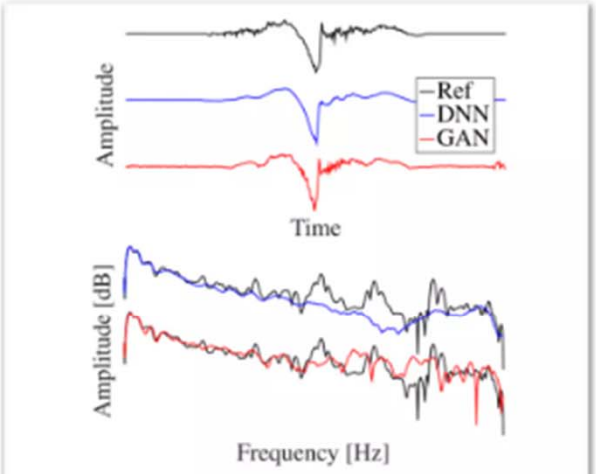
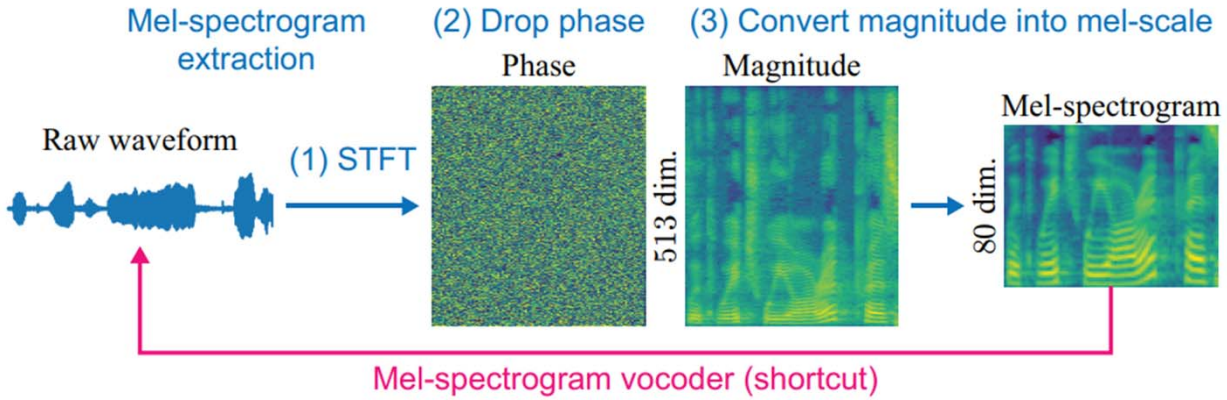


Fig. 2. Excitation pulses shown in the time and frequency domain. DNN pulse model (blue) fits the overall reference pulse (black) shape, but high-frequency stochastic components are smoothed out, whereas GAN (red) is able to generate realistic high-frequency components.

Ref: Figure from https://www.slideshare.net/jyamagis/advancements-in-neural-vocoders (page 113)

Mel-Vocoder

- **Mel-vocoder**: Mel-spectrogram → audio waveforms
  - Upsampling
  - Time-frequency conversion
    - Inverse the Mel-scale
    - Predict the phase



  - There is a "ground truth" output
  - The input ("Mel-spectrogram") is temporally aligned with the expected output
  - Paired data for training such a model can be easily obtained

Ref: Figure from https://arxiv.org/pdf/2203.02395.pdf

Source Separation vs Mel-Vocoder

Task	Input/Output	Conditioning type	Data availability
Source separation	audio (mixture) → audio (stem)	*strong & time-aligned* conditioning (perfect alignment)	Harder to get paired data
Mel-vocoder	mel → audio	*strong & time-aligned* conditioning (output sequence length is a factor of the input sequence length)	Extremely easy to get paired data

- *Strong* condition: it can be said that there is a "ground-truth" output
- *Time-aligned*: the condition is time-varying and is temporally aligned with the input
- From encoder/decoder perspective
  - Timbre classifier: encoder (downsampling)
  - Source separation : encoder/decoder
  - Mel-vocoder: decoder (upsampling)

Other Types of Audio Generation Problems

Task	Input/Output	Conditioning type	Data availability
Source separation	audio (mixture) → audio (stem)	*strong & time-aligned* conditioning	Harder to get paired data
Mel-vocoder	mel → audio	*strong & time-aligned* conditioning	Extremely easy to get paired data
Voice conversion	audio (source) → audio (target)		
Accomp. generation	audio (singing) → audio (accomp.)		
Audio synthesis	MIDI → audio		
Audio generation	x → audio		

Recap: Approaches to Deal with Phase for STFT-based Models

https://source-separation.github.io/tutorial/basics/phase.html

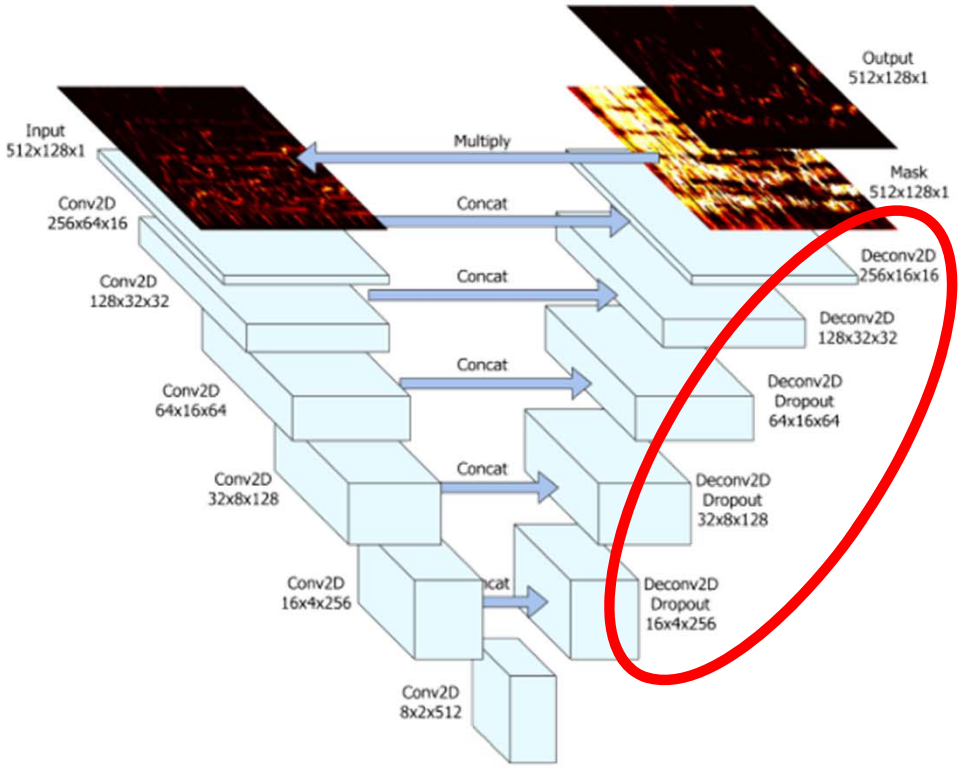
- **Copy the phase (from the input)**
  - Work fine for source separation (but NOT for Mel-vocoders)
- **Given the magnitude, estimate the phase**
  - Griffin & Lim
  - DL-based Mel-vocoders
- **Work on complex-valued spectrograms**
- **Work on audio waveforms, not spectrograms**

Outline

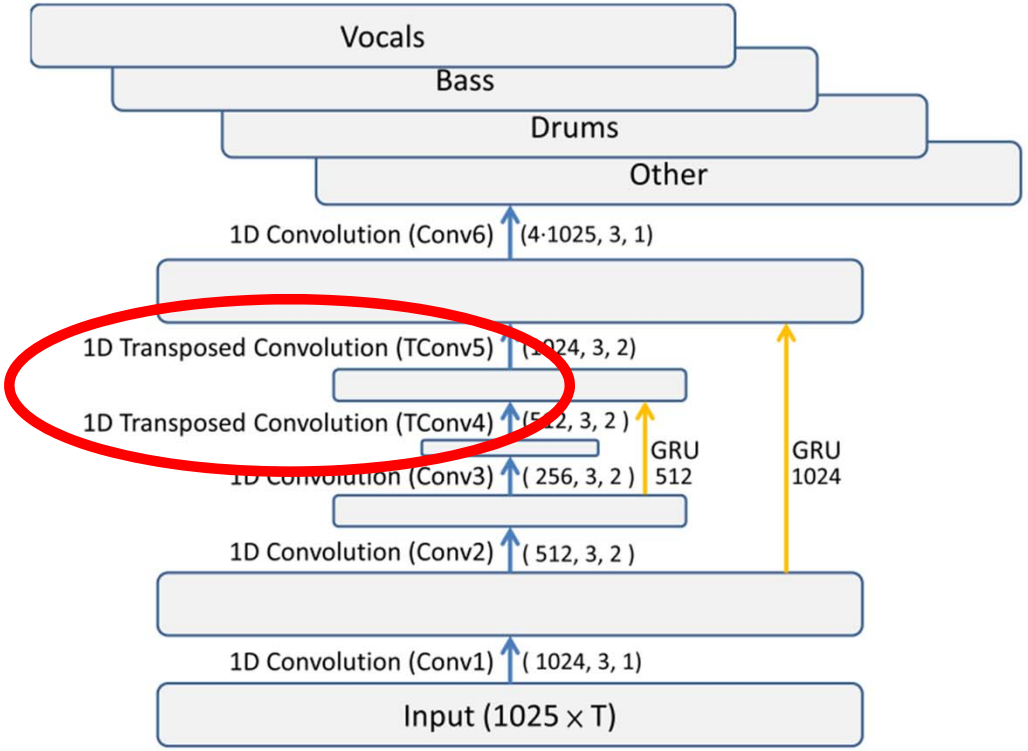
- General idea
- **DL components: upsampling layers and GAN**
- GAN-based Mel-vocoders

Upsampling Layers: Transposed Convolution

- Also known as "**deconvolution** layers"

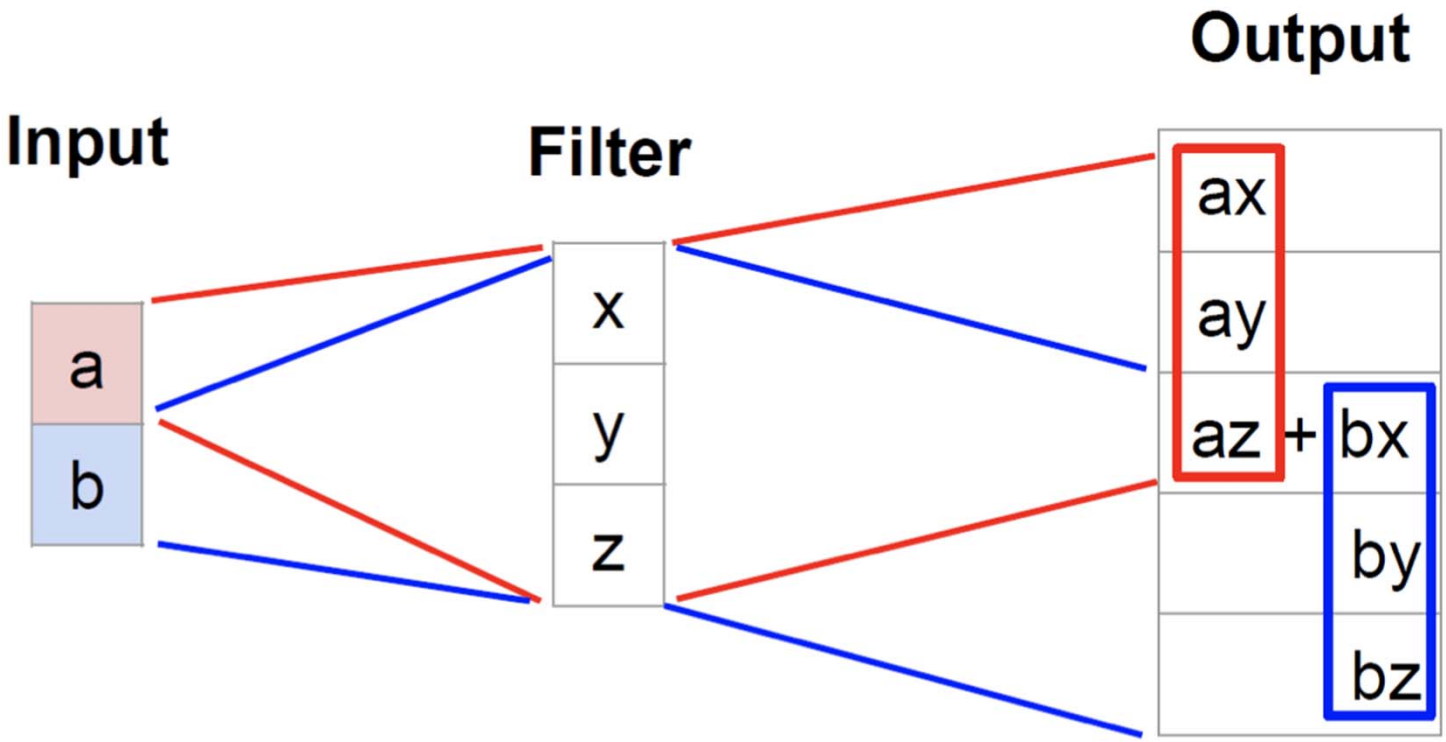


U-Net-based source separation model [jansson17ismir]



ARC [liu18icmla]

Convolution and Transposed Convolution



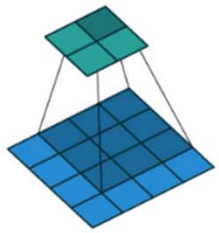
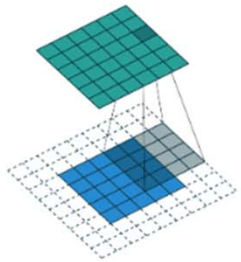
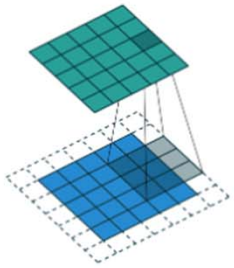
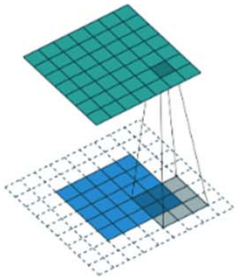
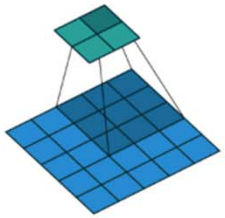
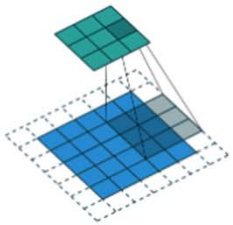
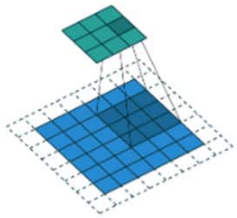
Ref: Figure from https://medium.com/jun94-devpblog/dl-12-unsampling-unpooling-and-transpose-convolution-831dc53687ce

Downsampling Layers: Convolution

https://github.com/vdumoulin/conv_arithmetic

Convolution animations

N.B.: Blue maps are inputs, and cyan maps are outputs.

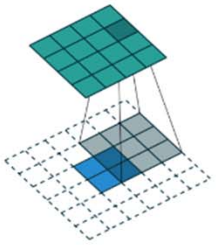
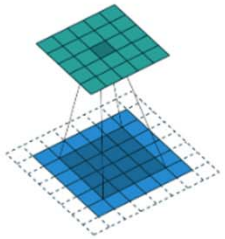
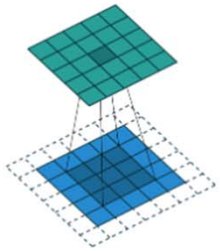
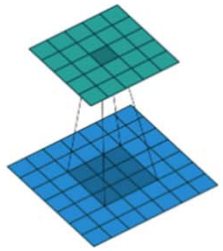
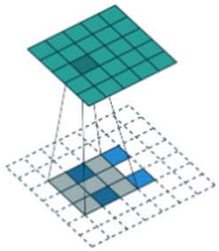
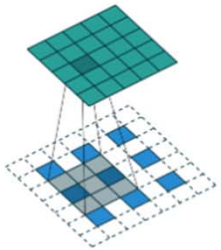
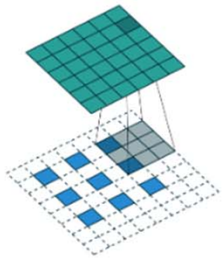
			
No padding, no strides	Arbitrary padding, no strides	Half padding, no strides	Full padding, no strides
			
No padding, strides	Padding, strides	Padding, strides (odd)	

Upsampling Layers: Transposed Convolution

https://github.com/vdumoulin/conv_arithmetic

Transposed convolution animations

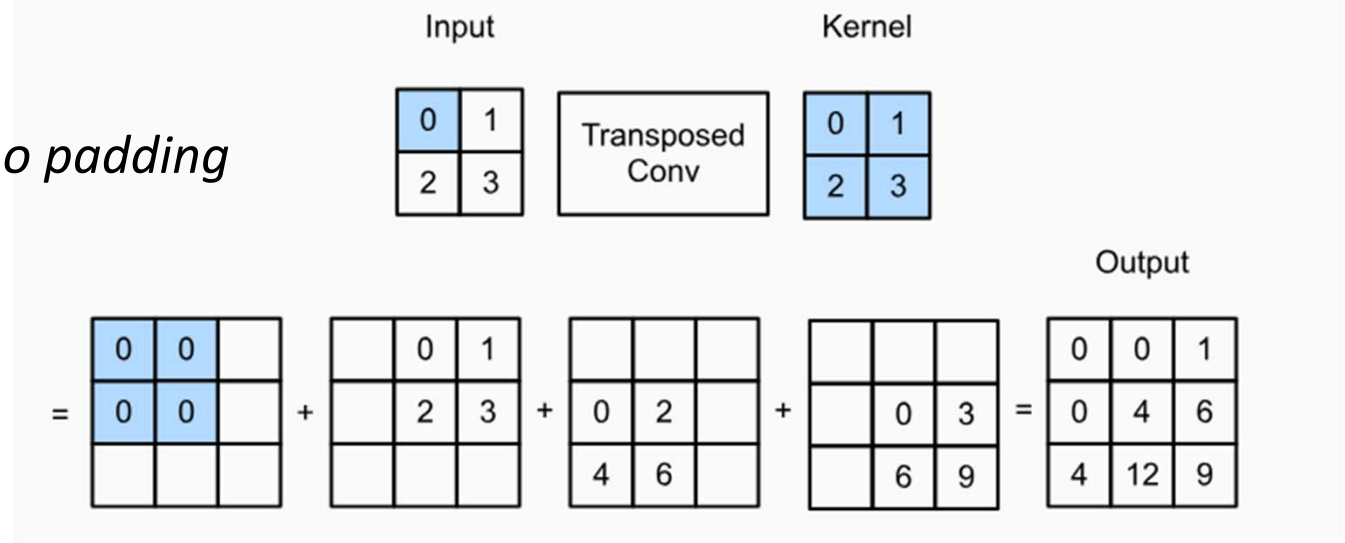
N.B.: Blue maps are inputs, and cyan maps are outputs.

			
No padding, no strides, transposed	Arbitrary padding, no strides, transposed	Half padding, no strides, transposed	Full padding, no strides, transposed
			
No padding, strides, transposed	Padding, strides, transposed	Padding, strides, transposed (odd)	

Upsampling Layers: Transposed Convolution

https://d2l.ai/chapter_computer-vision/transposed-conv.html

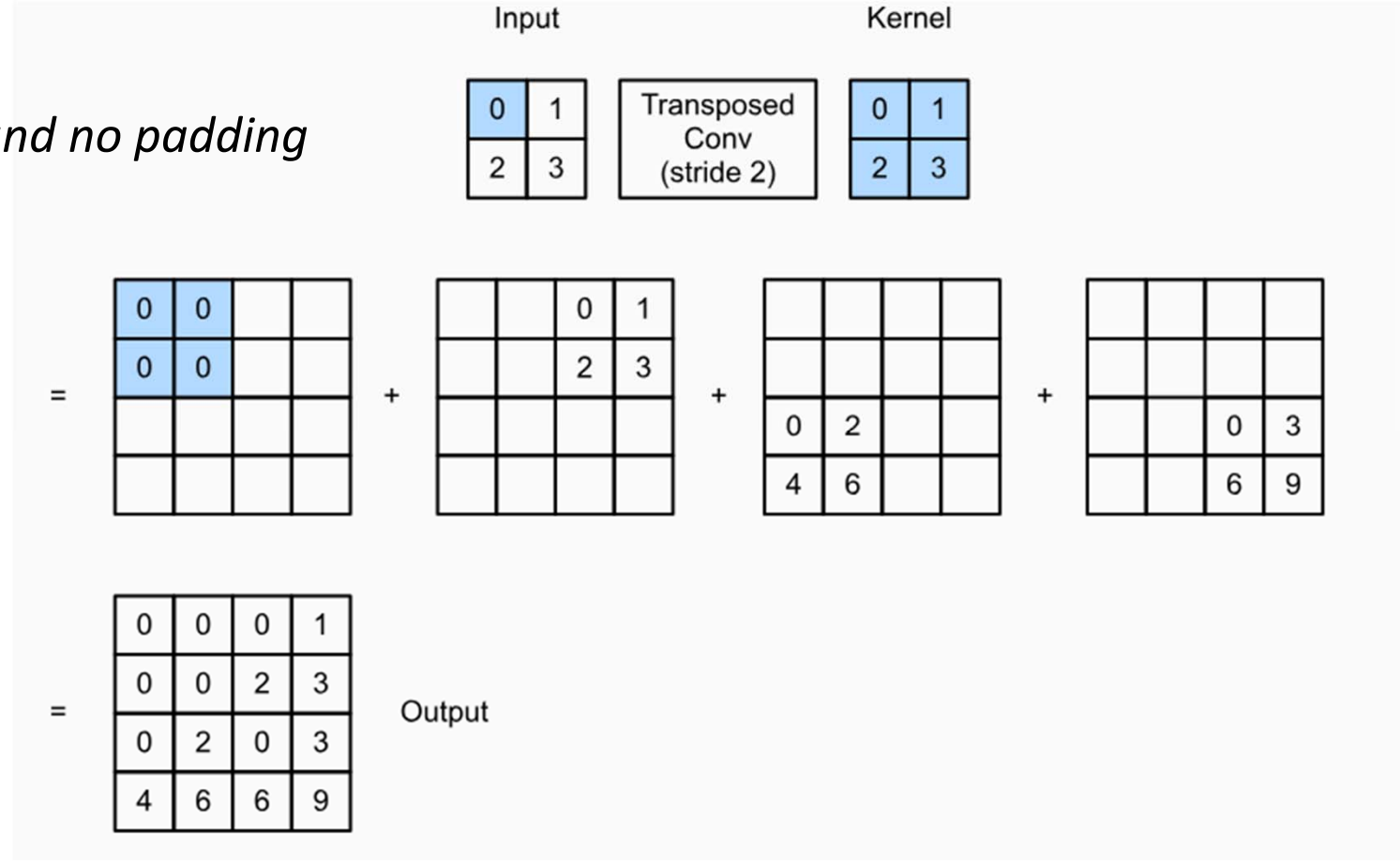
*stride of **1** and no padding*



Upsampling Layers: Transposed Convolution

https://d2l.ai/chapter_computer-vision/transposed-conv.html

*stride of **2** and no padding*

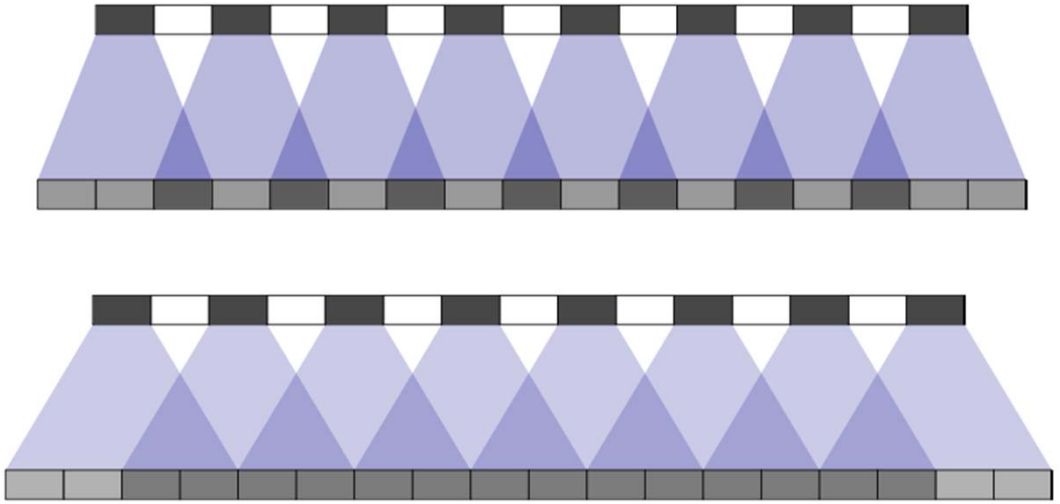


Upsampling Layers: The Checkerboard Artifacts in Images

https://distill.pub/2016/deconv-checkerboard/

- Due to uneven overlaps: the kernel size (the output window size) is not divisible by the stride (the spacing between points on the top)
- Solution: Use a kernel size that is divided by your stride

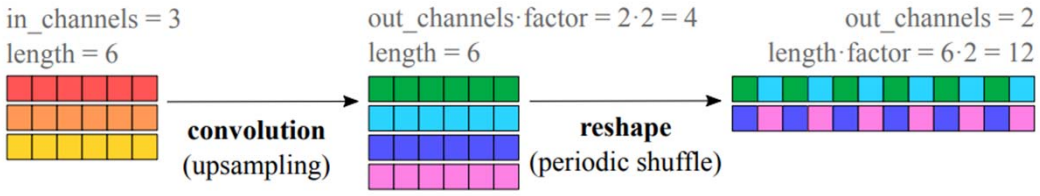




Upsampling Layers: The Checkerboard Artifacts in Audio

http://www.jordipons.me/apps/upsamplers/

- Explore other neural upsampler approaches
  - **Transposed convolution**: no overlap (length=stride) or full overlap (length is multiple of the stride), not partial overlap
    - "Even under no overlap and full overlap setups, the loss function and the weights' initialization issues remain"
  - **Interpolation upsamplers**
    - Nearest neighbor interpolation
    - Linear interpolation
    - Stretch interpolation
    - Sinc interpolation
  - **Subpixel convolution**



Ref: Pons et al, "Upsampling layers for music source separation," EUSIPCO 2023

Upsampling Layers: The Checkerboard Artifacts in Audio

http://www.jordipons.me/apps/upsamplers/

- Explore other neural upsampler approaches
- In the context of **source separation**

Music source separation (MUSDB [27] benchmark)	SDR ↑	epoch	#parm
Demucs-like (Fig. 1, b): transposed CNN (full-overlap)	5.35	319 s	703M
Demucs-like (Fig. 1, c): nearest neighbor interpolation	5.17	423 s	716M
Demucs-like: linear interpolation	4.62	430 s	716M
Demucs-like (Fig. 1, d): subpixel CNN	5.38	311 s	729M
Modified (Fig. 9, a): transposed CNN (full-overlap)	5.37	326 s	703M
Modified (Fig. 9, b): subpixel CNN	5.38	315 s	729M
WaveUnet [2]: linear intepolation	3.23	-	10M
Demucs [3]: transposed convolution (full-overlap)	5.34[4]	-	648M
OpenUnmix [30]: spectrogram-based	5.36	-	8.9M
Sams-net [31]: spectrogram-based	5.65	-	3.7M

"Informal listening, however, reveals that **tonal artifacts** can emerge even after training, **especially in silent parts and with out-of-distribution data** (e.g., with sounds and conditions not seen during training)"

"We find that **nearest neighbor and linear interpolation models do not have this disadvantage, although they achieve worse SDR scores**."

Ref: Pons et al, "Upsampling artifacts in neural audio synthesis," ICASSP 2021

Upsampling Layers “for Waveforms”: Snake Activation

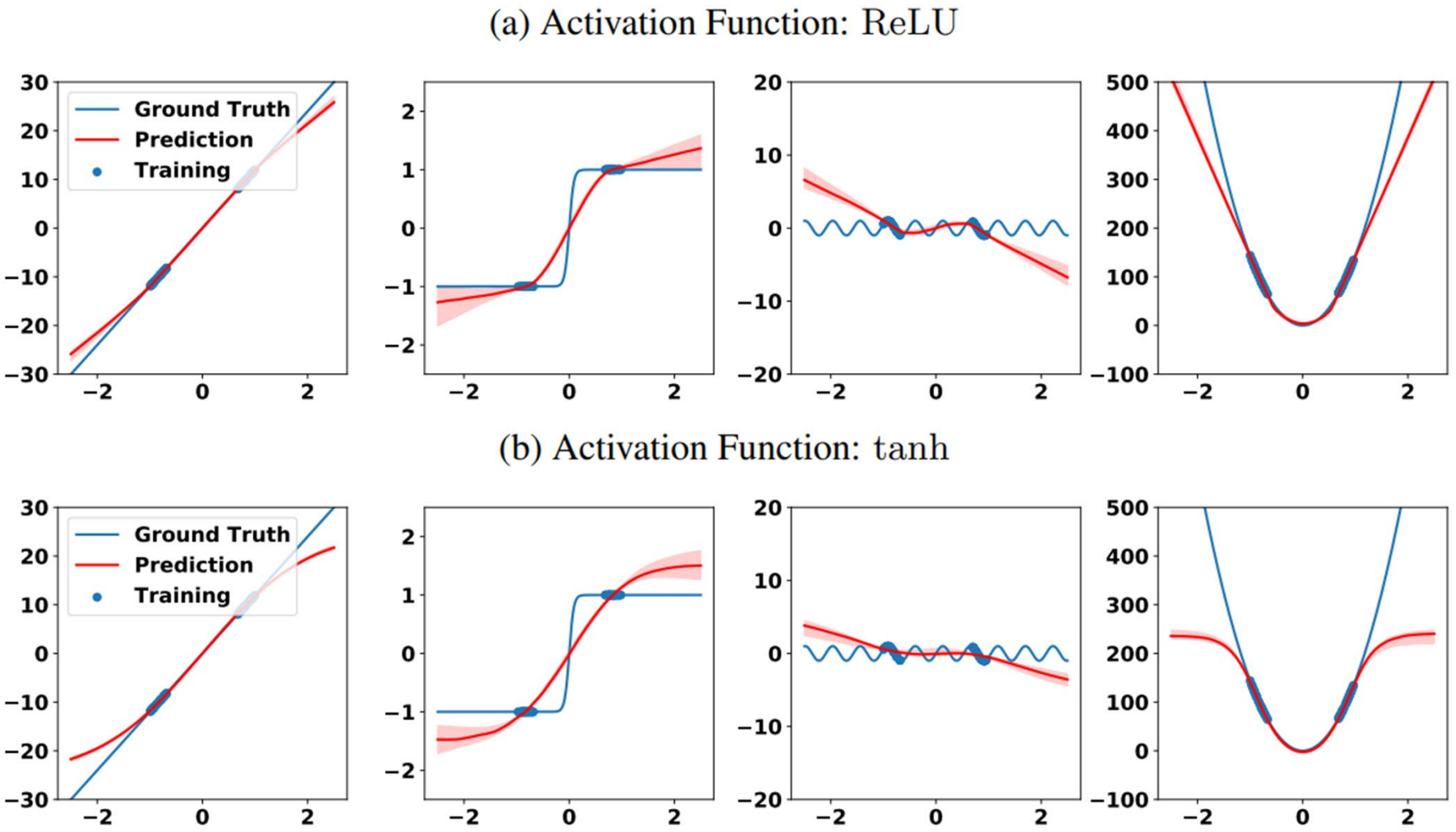
- **Mel-vocoder**: Mel-spectrogram → audio waveforms

Neural Networks Fail to Learn Periodic Functions and How to Fix It

Liu Ziyin[1], **Tilman Hartwig**[1,2,3], **Masahito Ueda**[1,2,4]
[1]Department of Physics, School of Science, The University of Tokyo
[2]Institute for Physics of Intelligence, School of Science, The University of Tokyo
[3]Kavli IPMU (WPI), UTIAS, The University of Tokyo
[4]RIKEN CEMS

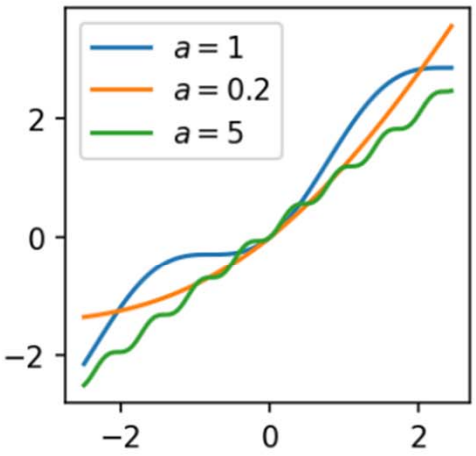
34th Conference on Neural Information Processing Systems (NeurIPS 2020), Vancouver, Canada.

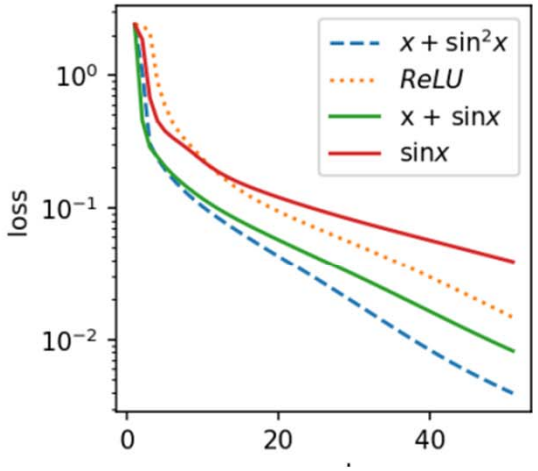
Upsampling Layers “for Waveforms”: Snake Activation

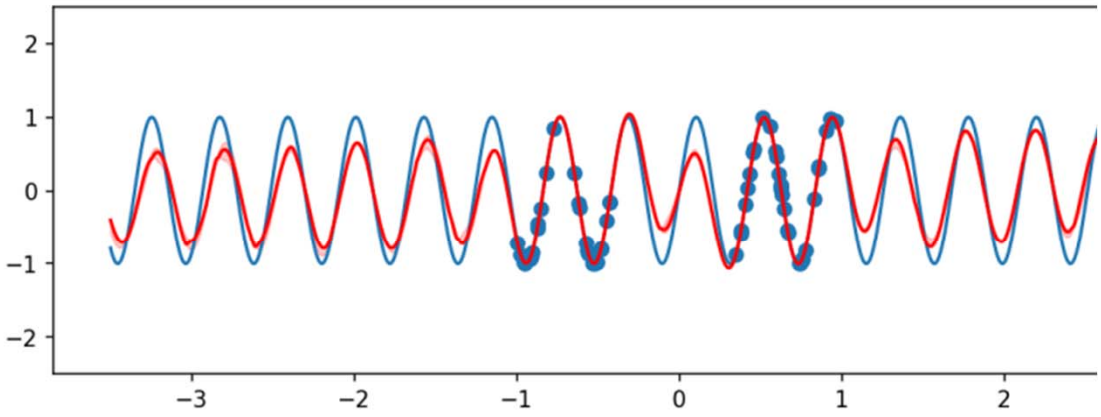


Upsampling Layers “for Waveforms”: Snake Activation

$$\text{Snake}_a := x + \frac{1}{a}\sin^2(ax) = x - \frac{1}{2a}\cos(2ax) + \frac{1}{2a},$$





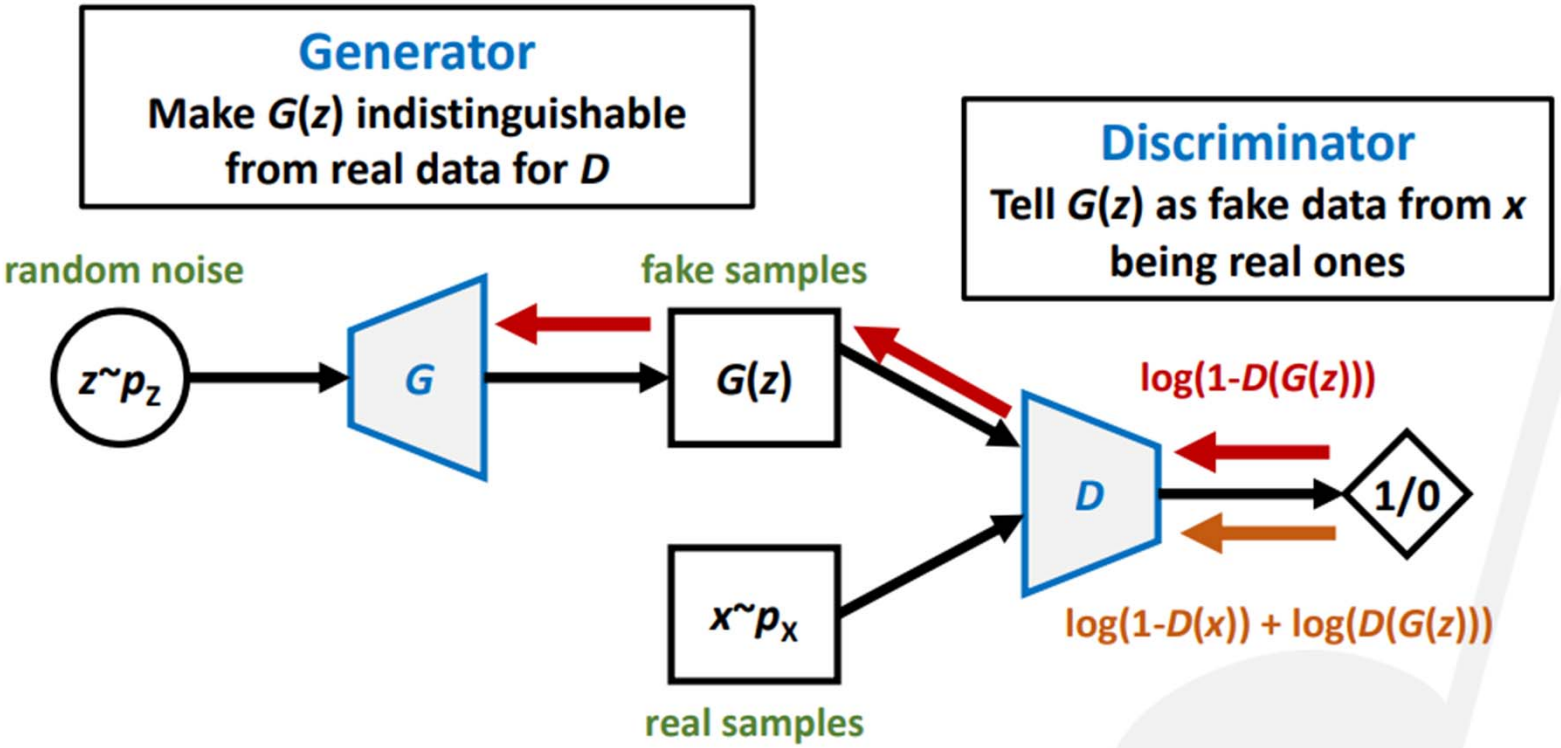


	ReLU	Swish	Tanh	$\sin(x)$	$x + \sin(x)$	$x + \sin^2(x)$
monotonic	✓	✗	✓	✗	✓	✓
(semi-)periodic	✗	✗	✗	✓	✓	✓
first non-linear term	-	$\frac{x^2}{4}$	$-\frac{x^3}{3}$	$-\frac{x^3}{6}$	$-\frac{x^3}{6}$	x^2

Table 1: Comparison of different periodic and non-periodic activation functions.

GAN: Generative Adversarial Network

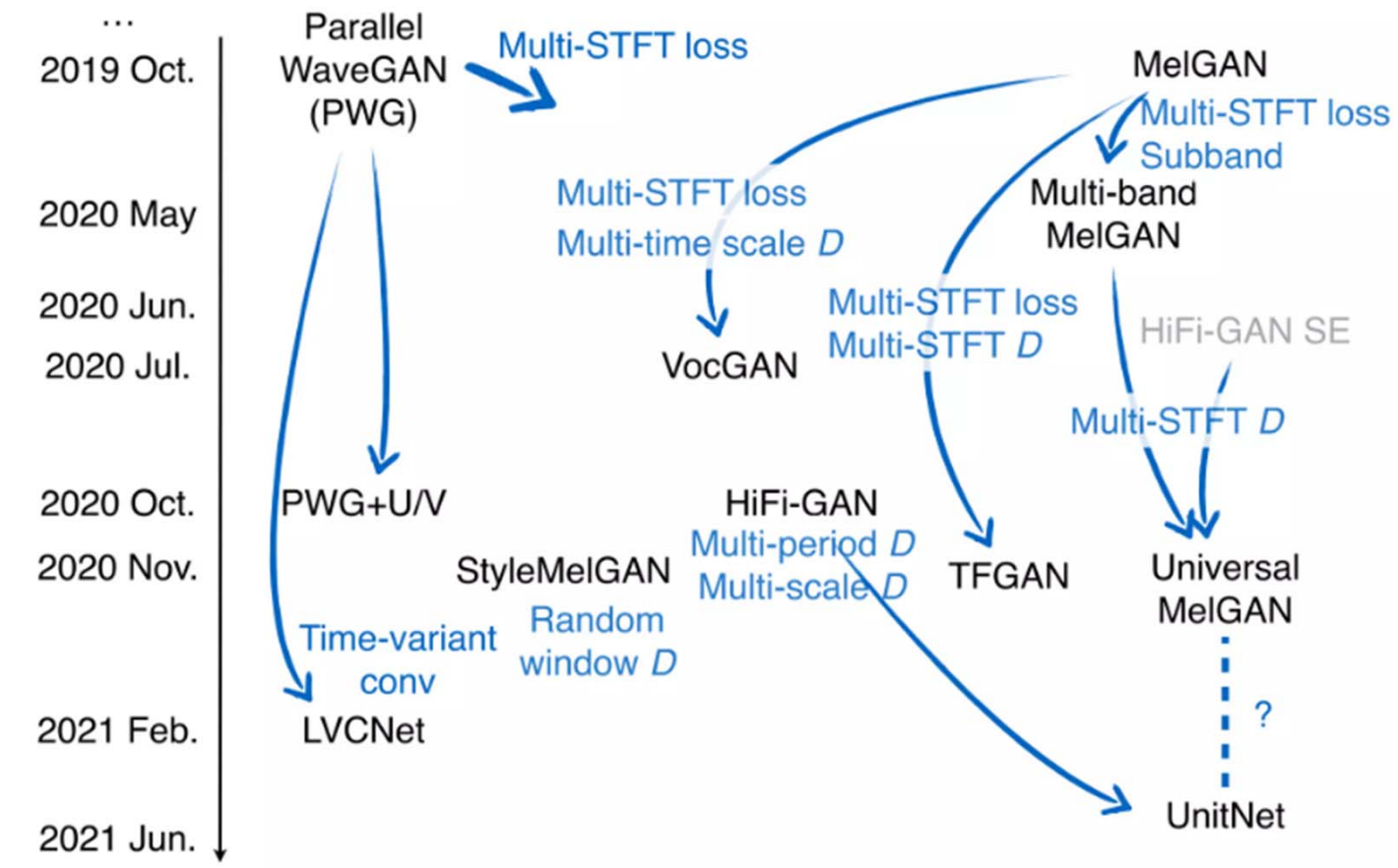
https://salu133445.github.io/ismir2019tutorial/



Outline

- General idea
- DL components: upsampling layers and GAN
- **GAN-based Mel-vocoders**
- More recent work

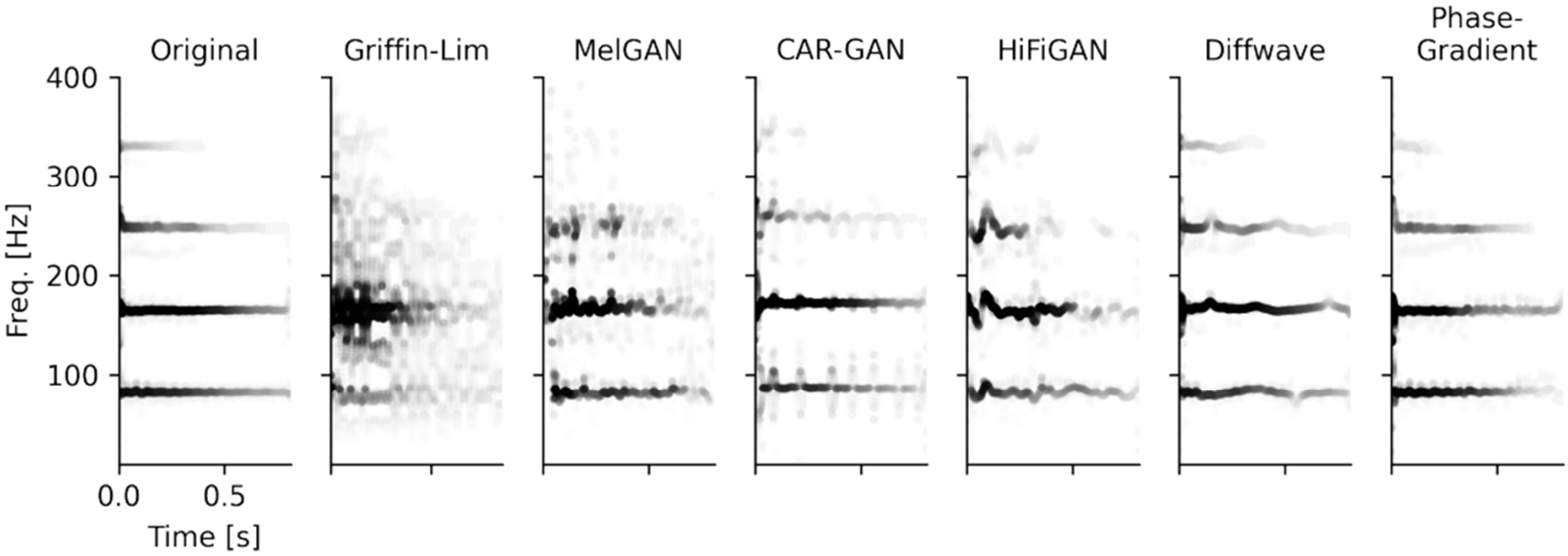
GAN-based Models



Ref: Figure from https://www.slideshare.net/jyamagis/advancements-in-neural-vocoders (page 112)

GAN-based Models

- Great improvement over the years



Ref: Di Giorgi et al., “Mel spectrogram inversion with stable pitch,” ISMIR 2022

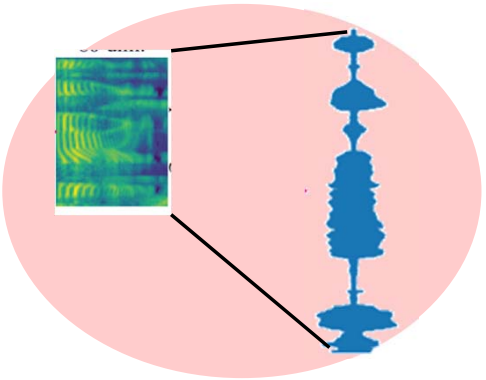
GAN-based Models

- Let's talk about the following models
  - MelGAN (arXiv:1910.06711; NeurIPS'19)
  - MB-MelGAN (arXiv:2005.05106; INTERSPEECH'20)
  - HiFiGAN (arXiv:2010.05646; NeurIPS'20)
  - iSTFTNet (arXiv:2203.02395; ICASSP'22)

MelGAN (NeurIPS'19)

https://github.com/descriptinc/melgan-neurips/blob/master/melgan_slides.pdf

- First GAN-based, non-autoregressive Mel-Vocoder
  - ***Feedforward***: upsamples the input to create the output
- Smaller and **much faster**



Model	Number of parameters (in millions)	Speed on CPU (in kHz)	Speed on GPU (in kHz)
Wavenet (Shen et al., 2018)	24.7	0.0627	0.0787
Clarinet (Ping et al., 2018)	10.0	1.96	221
WaveGlow (Prenger et al., 2019)	87.9	1.58	223
MelGAN (ours)	**4.26**	**51.9**	**2500**

Ref: Kumar et al., "MelGAN: Generative adversarial networks for conditional waveform synthesis," NeurIPS 2019

MelGAN

https://github.com/descriptinc/melgan-neurips/blob/master/melgan_slides.pdf

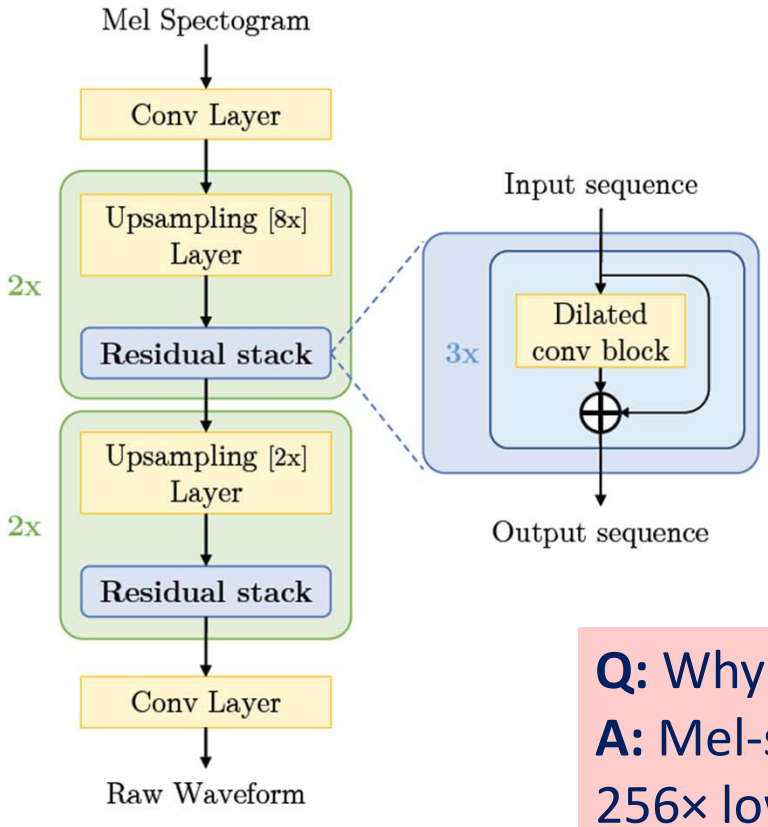
- First GAN-based, non-autoregressive Mel-Vocoder
- Smaller and much faster than competing methods at the time
- But a bit less accurate than WaveNet and flow-based models such as WaveGlow

Model	MOS	95% CI
Griffin Lim	1.57	±0.04
WaveGlow	4.11	±0.05
WaveNet	4.05	±0.05
MelGAN	3.61	±0.06
Original	**4.52**	± **0.04**

Ref: Kumar et al., "MelGAN: Generative adversarial networks for conditional waveform synthesis," NeurIPS 2019

MelGAN: Generator

https://github.com/descriptinc/melgan-neurips/blob/master/melgan_slides.pdf



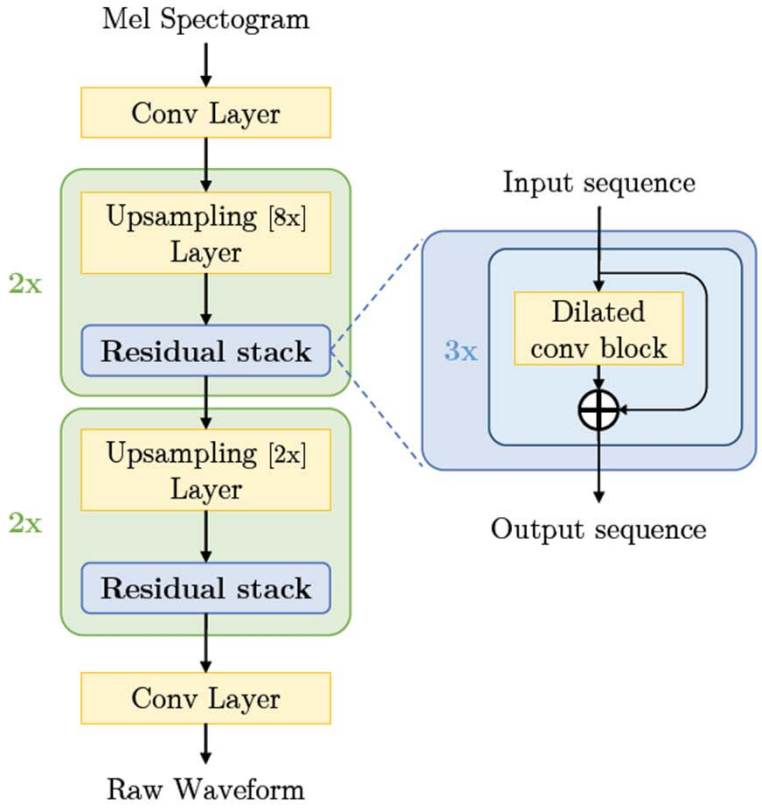
Architecture

- Stack of transposed convolutional layers to upsample the input sequence.
- Each transposed convolutional layer followed by a stack of residual blocks.

Q: Why [8x8x2x2]?
A: Mel-spectrogram (used for all experiments) is at a 256× lower temporal resolution

MelGAN: Generator

https://github.com/descriptinc/melgan-neurips/blob/master/melgan_slides.pdf

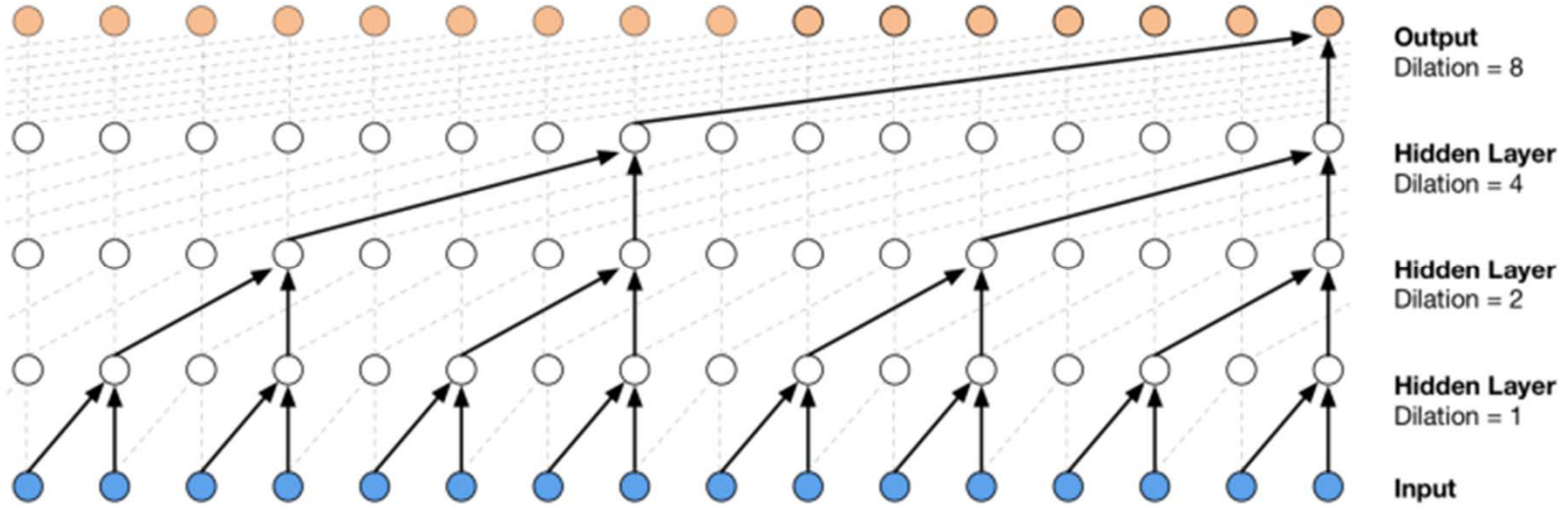


Induced Receptive Field

- **Residual blocks with dilations** so temporally far output activations of each layer has significant overlapping inputs.
- Receptive field of a stack of dilated convolution layers **increases exponentially** with the number of layers.

Dilation?

- **Dilated convolution** (≠ transposed convolution)
  - Increase reception field exponentially with linearly increasing number of parameters
  - See the WaveNet paper
- Stride ≠ dilation



Ref: van den Oord et al, "Wavenet: A generative model for raw audio," ISCA 2016

MelGAN: Generator

https://github.com/descriptinc/melgan-neurips/blob/master/melgan_slides.pdf

Checkerboard Artifacts

- Kernel-size as a **multiple** of stride
- Dilation grows as a **power** of the kernel-size
- Receptive field of the stack looks like a fully balanced (seeing input uniformly) and symmetric tree

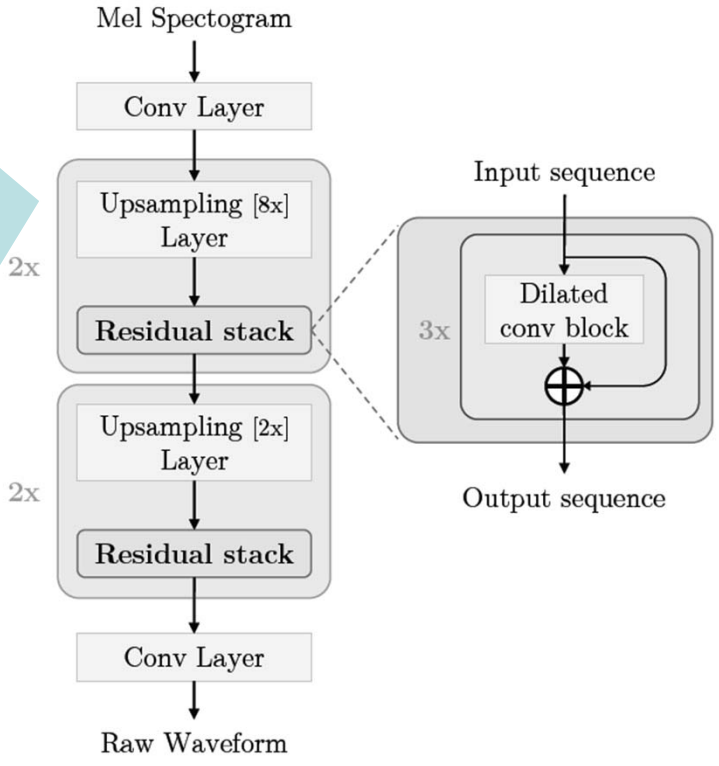
Normalization

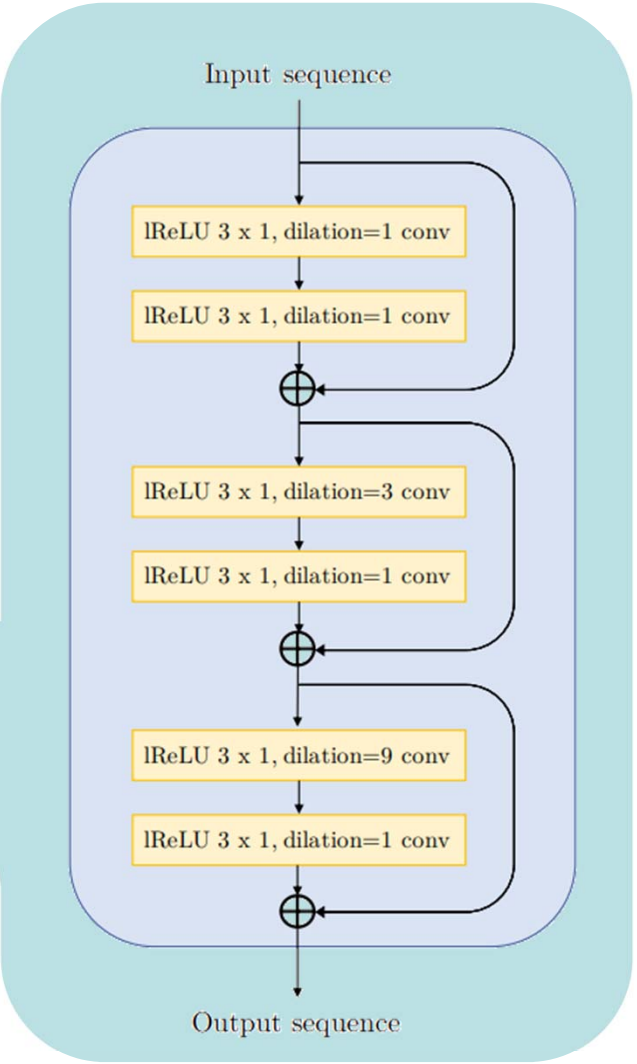
- Instance-norm washes away pitch information, making audio sound metallic
- Spectral-norm's strong Lipshitz constraint badly impacts feature matching objective
- **Weight-norm** works best

MelGAN: Generator

7×1, stride=1 conv 512
lReLU 16×1, stride=8 conv transpose 256
Residual Stack 256
lReLU 16×1, stride=8 conv transpose 128
Residual Stack 128
lReLU 4×1, stride=2 conv transpose 64
Residual Stack 64
lReLU 4×1, stride=2 conv transpose 32
Residual Stack 32
lReLU 7×1, stride=1 conv 1 Tanh

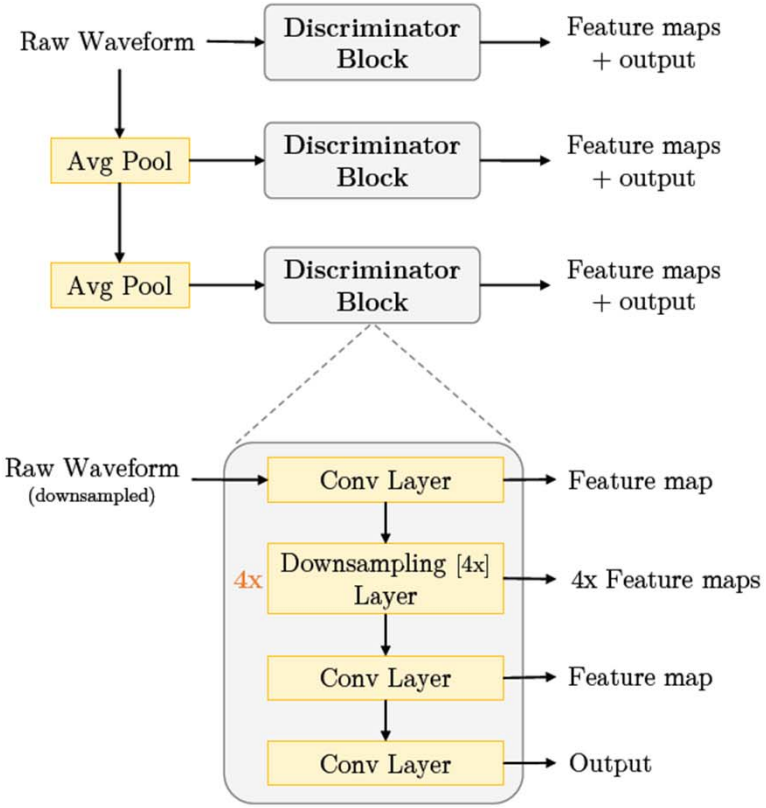
(a) Generator Architecture





MelGAN: Discriminator

https://github.com/descriptinc/melgan-neurips/blob/master/melgan_slides.pdf



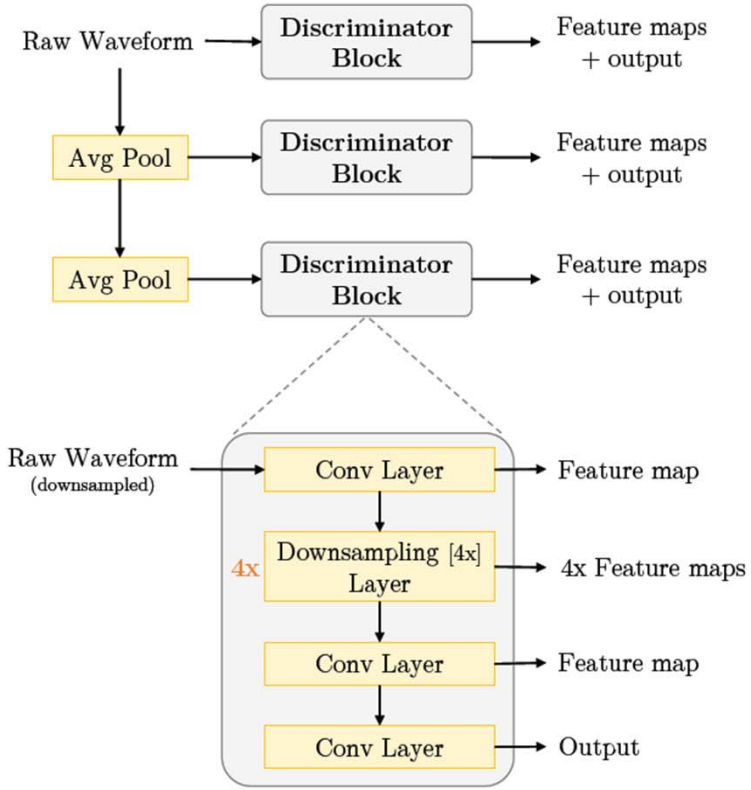
Multiscale Architecture

- 3 discriminators (identical structure) operate on different audio scales -- original scale, 2x and 4x downsampled.
- Each discriminator biased to learn features for **different frequency range** of the audio.

"For example, the discriminator operating on downsampled audio does not have access to high frequency component, hence, it is biased to learn discriminative features based on low frequency components only."

MelGAN: Discriminator

https://github.com/descriptinc/melgan-neurips/blob/master/melgan_slides.pdf



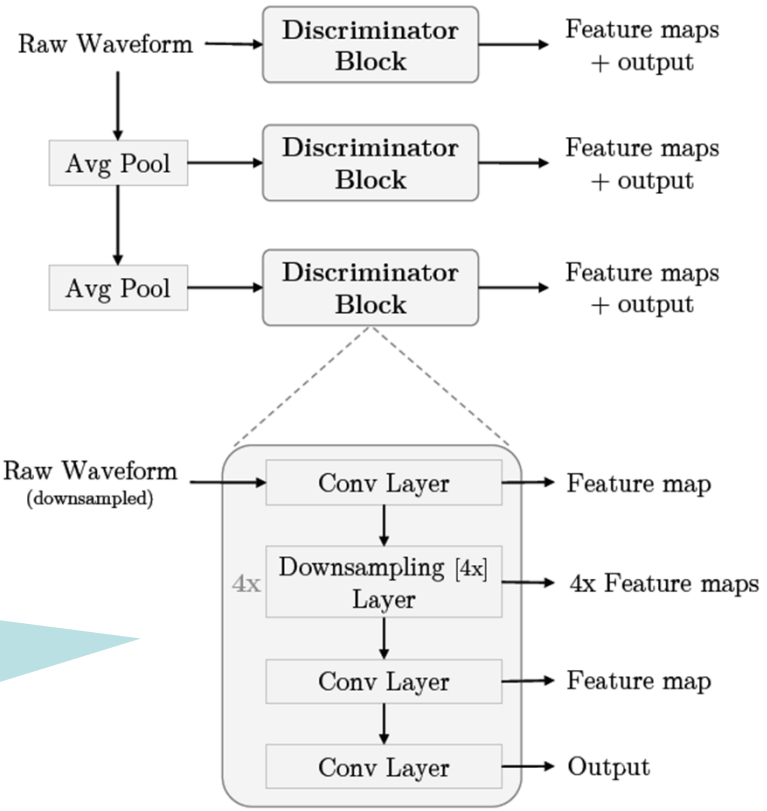
Window-based objective

- Each individual discriminator is a Markovian window-based discriminator (analogues to image patches, Isola et al. (2017))
- Discriminator learns to classify between distributions of **small audio chunks**.
- Overlapping large windows maintain coherence across patches

MelGAN: Discriminator

15×1, stride=1 conv 16 lReLU
41×1, stride=4 groups=4 conv 64 lReLU
41×1, stride=4 groups=16 conv 256 lReLU
41×1, stride=4 groups=64 conv 1024 lReLU
41×1, stride=4 groups=256 conv 1024 lReLU
5×1, stride=1 conv 1024 lReLU
3×1, stride=1 conv 1

(b) Discriminator Block Architecture



MelGAN: Loss Functions

https://github.com/descriptinc/melgan-neurips/blob/master/melgan_slides.pdf

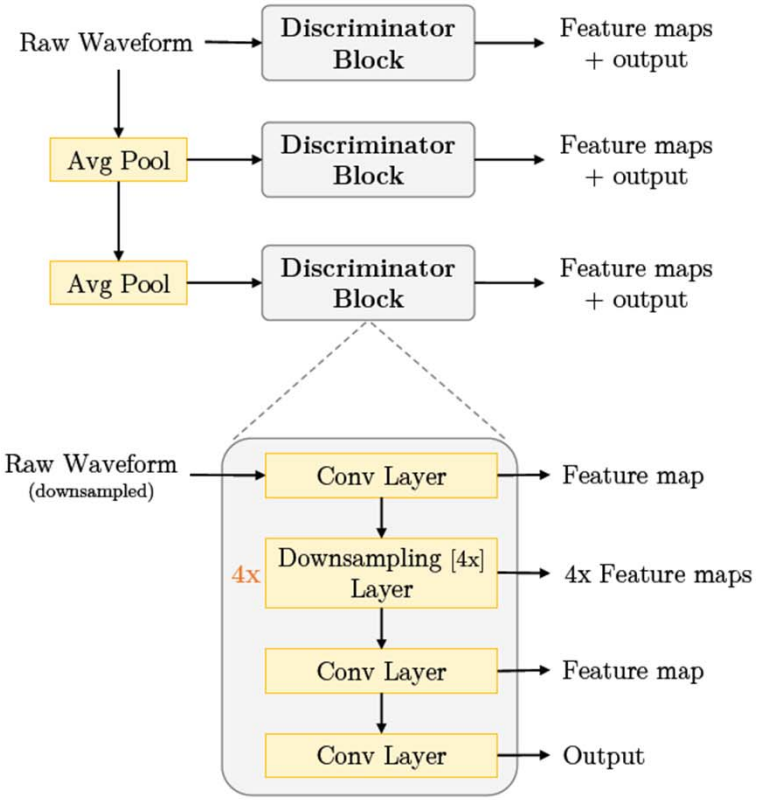
We use the hinge loss formulation (Lim & Ye, 2017; Miyato et al., 2018):

$$\min_{D_k} \mathbb{E}_x \Big[\min(0, 1 - D_k(x)) \Big] + \mathbb{E}_{s,z} \Big[\min(0, 1 + D_k(G(s,z))) \Big], \ \forall k = 1, 2, 3$$

$$\min_{G} \mathbb{E}_{s,z} \left[\sum_{k=1,2,3} -D_k(G(s,z)) \right]$$

MelGAN: Loss Functions

https://github.com/descriptinc/melgan-neurips/blob/master/melgan_slides.pdf



We additionally use a feature matching objective (Larsen et al., 2015) to train the generator:

$$\mathcal{L}_{\text{FM}}(G, D_k) = \mathbb{E}_{x,s\sim p_{\text{data}}}\left[\sum_{i=1}^{T}\frac{1}{N_i}||D_k^{(i)}(x) - D_k^{(i)}(G(s))||_1\right]$$

Overall Generator Objective:

$$\min_G\left(\mathbb{E}_{s,z}\left[\sum_{k=1,2,3} -D_k(G(s,z))\right] + \lambda\sum_{k=1}^{3}\mathcal{L}_{\text{FM}}(G, D_k)\right)$$

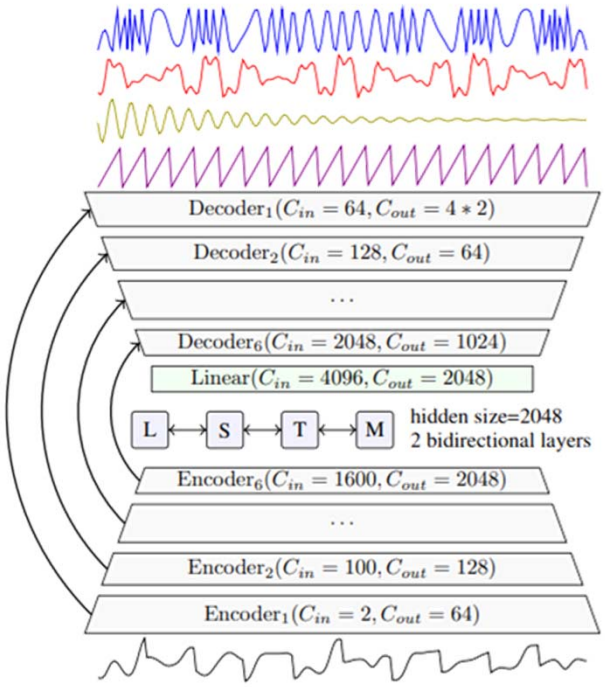
Question: Why Not Waveform-domain Loss Functions?

https://www.soundsandwords.io/audio-loss-functions/

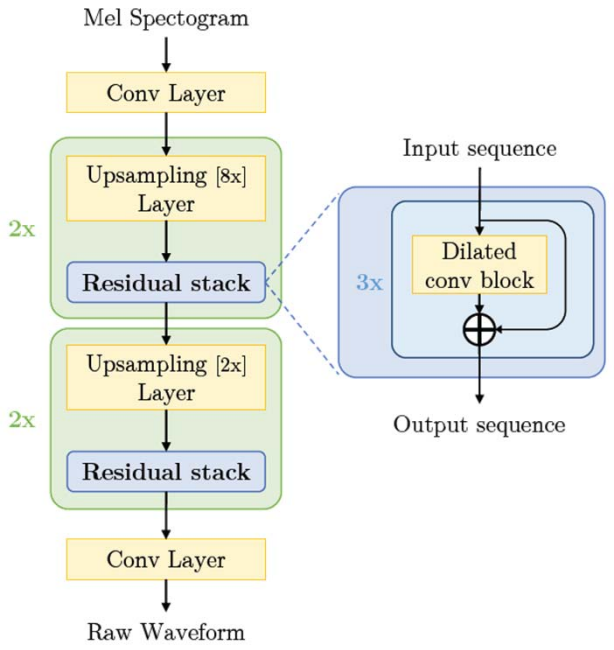
- **Not shift invariant**
  - "If your models comes up with the exact correct output, but it's shifted just 1 millisecond too early or late, a human would consider it a perfect match but error would likely spike. This can especially manifest as phase shift invariance, where two waves have similar frequencies but out-of-sync phases, leading to a large gap in waveform distance."
- Not reflect our **perception** of audio
  - "If training a model for human ears, with L1/L2 you risk overweighting the importance of low frequency sounds"

Question: Why not U-Net?

- **Source separation**
  - Use U-Net
  - Use waveform domain loss

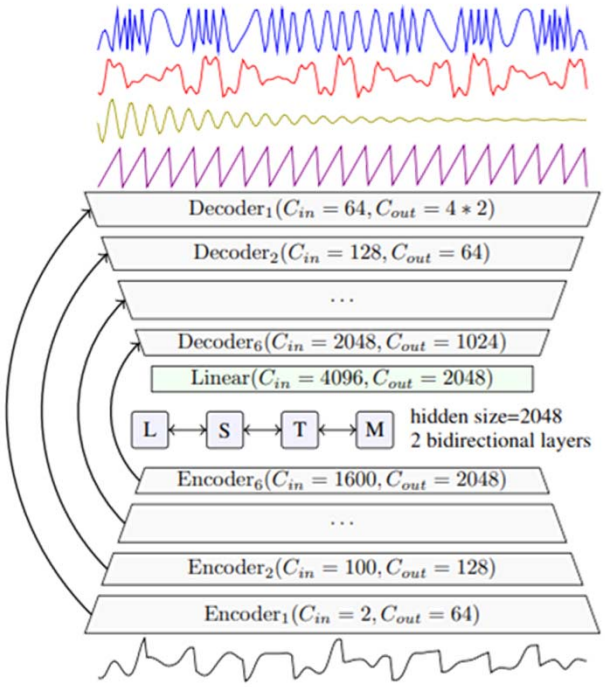


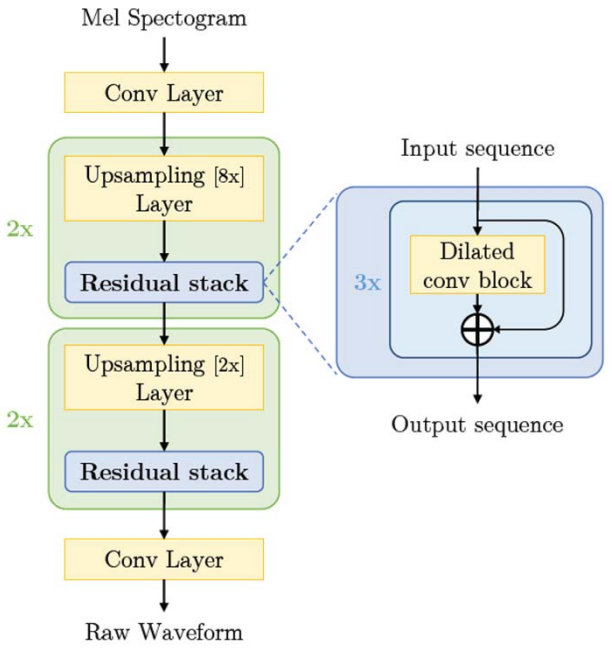
- **Mel-vocoder**
  - Do not use U-Net
  - Dot not use waveform domain loss



Question: Why Not GAN for Source Separation?

- **Source separation**
  - Seldom use GAN
- **Mel-vocoder**
  - Use GAN a lot





MelGAN: Ablation Study

Model	MOS	95% CI
w/ Spectral Normalization	1.33	±0.07
w/ L1 loss (audio space)	2.59	±0.11
w/o Window-based Discriminator	2.29	±0.10
w/o Dilated Convolutions	2.60	±0.10
w/o Multi-scale Discriminator	2.93	±0.11
w/o Weight Normalization	3.03	±0.10
Baseline (MelGAN)	**3.09**	± **0.11**

- "Instance normalization washes away important pitch information, making the audio sound metallic. We also obtained poor results when applying spectral normalization"
- "Adding L1 loss in audio space introduces audible noise that hurts audio quality"

Ref: Kumar et al., "MelGAN: Generative adversarial networks for conditional waveform synthesis," NeurIPS 2019

MB-MelGAN

- Improved version of MelGAN

Index	Model	MOS
F0	MelGAN [20]	3.98±0.04
F1	+ Pretrain G	4.04±0.03
F2	+ $L_{mr_stft}(G)$	4.06±0.04
F3	+ Deepen ResStack	**4.35±0.05**

1. Increase the receptive field of the generator (F3)
2. Substitute the feature matching loss with the **multi-resolution STFT loss** (F1, F2)
   - Avoid the problems of **time-domain loss**
   - **Pre-train** the generator using STFT loss, then adversarial loss
3. Extend MelGAN with **multiband processing**
   - The generator takes mel-spectrograms as input and produces sub-band signals which are subsequently summed back to full-band signals as discriminator input.

Ref: Yang et al, "Multi-band MelGAN: Faster Waveform Generation for High-Quality Text-to-Speech," INTERSPEECH 2020

Multi-resolution STFT Loss

- Firstly explored elsewhere (e.g., in Parallel WaveGAN)
- Avoid the problems of time-domain loss
- **Multiple resolutions**: different window sizes and hop sizes

$$L_{sc}(x, \widetilde{x}) = \frac{\| |STFT(x)| - |STFT(\widetilde{x})| \|_F}{\| |STFT(x)| \|_F}$$

$$L_{mag}(x, \widetilde{x}) = \frac{1}{N} \| log|STFT(x)| - log|STFT(\widetilde{x})| \|_1$$

$$L_{mr_stft}(G) = \mathbb{E}_{x,\widetilde{x}}[\frac{1}{M} \sum_{m=1}^{M} (L_{sc}^m(x, \widetilde{x}) + L_{mag}^m(x, \widetilde{x}))]$$

- **Pre-train** the generator using STFT loss, then adversarial loss

HiFiGAN (NeurIPS'20)

https://github.com/jik876/hifi-gan

- A great improved version of MelGAN that outperforms WaveNet, WaveGlow
- Widely-used

Model	MOS (CI)	Speed on CPU (kHz)	Speed on GPU (kHz)	# Param (M)
Ground Truth	4.45 (±0.06)	–	–	–
WaveNet (MoL)	4.02 (±0.08)	–	0.07 (×0.003)	24.73
WaveGlow	3.81 (±0.08)	4.72 (×0.21)	501 (×22.75)	87.73
MelGAN	3.79 (±0.09)	145.52 (×6.59)	14,238 (×645.73)	4.26
Highest MOS → HiFi-GAN $V1$	**4.36** (±0.07)	31.74 (×1.43)	3,701 (×167.86)	13.92
Most light-weight → HiFi-GAN $V2$	4.23 (±0.07)	214.97 (×9.74)	16,863 (×764.80)	**0.92**
Fastest → HiFi-GAN $V3$	4.05 (±0.08)	**296.38 (×13.44)**	**26,169 (×1,186.80)**	1.46

Ref: Kong et al., "HiFi-GAN: Generative adversarial networks for efficient and high fidelity speech synthesis," NeurIPS 2020

HiFiGAN: Generator

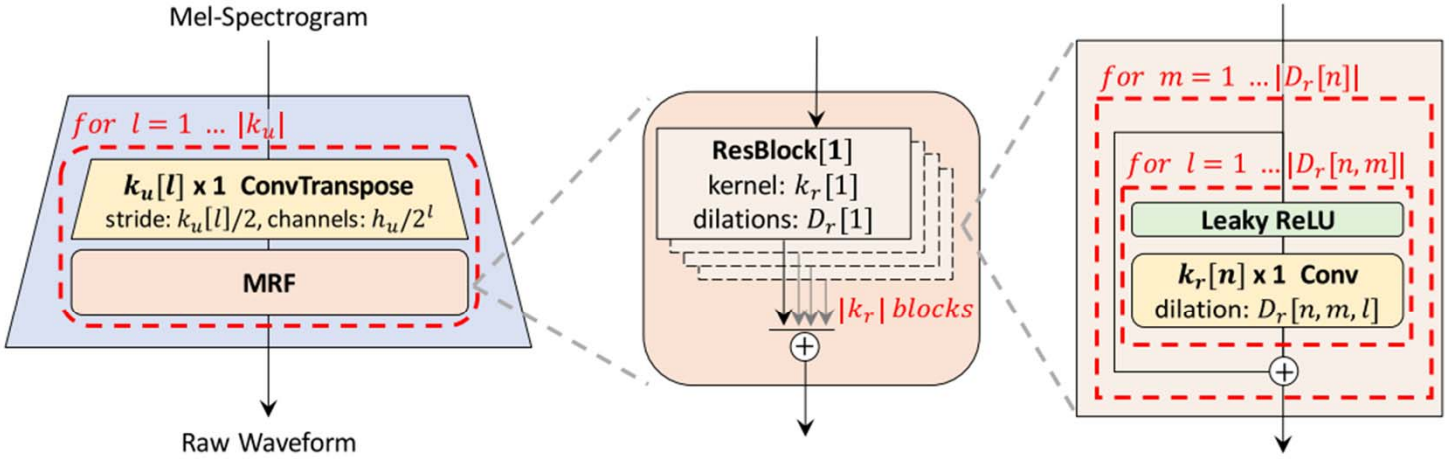


Figure 1: The generator upsamples mel-spectrograms up to $|k_u|$ times to match the temporal resolution of raw waveforms. A MRF module adds features from $|k_r|$ residual blocks of different kernel sizes and dilation rates. Lastly, the n-th residual block with kernel size $k_r[n]$ and dilation rates $D_r[n]$ in a MRF module is depicted.

- Multi-receptive field fusion (MRF)
- Different **kernel sizes** and **dilation rates** are selected for each **residual block** to form diverse receptive field patterns

HiFiGAN: Generator

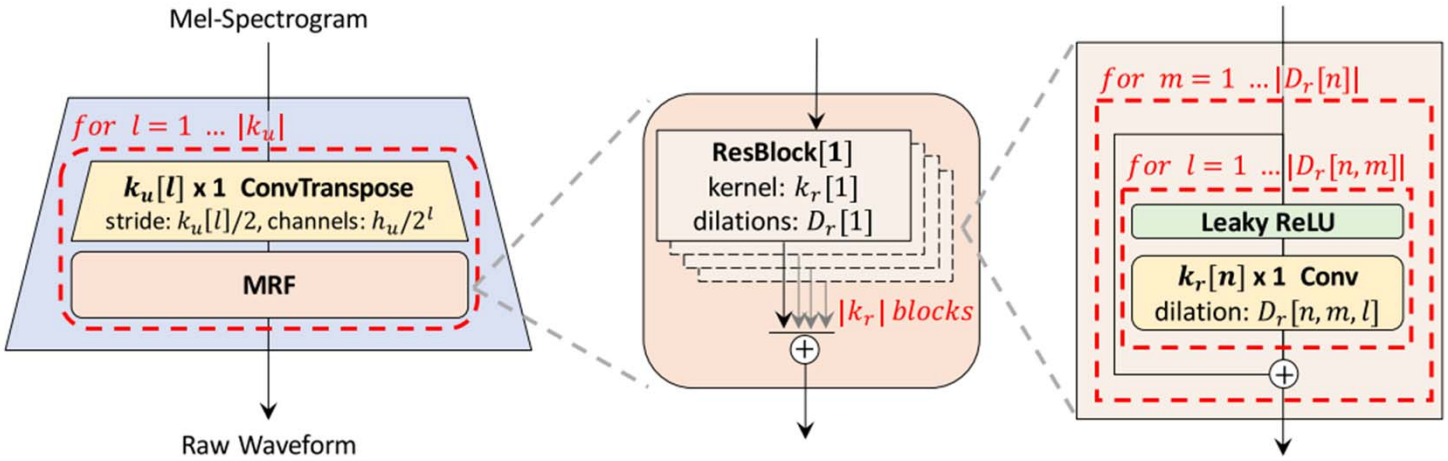
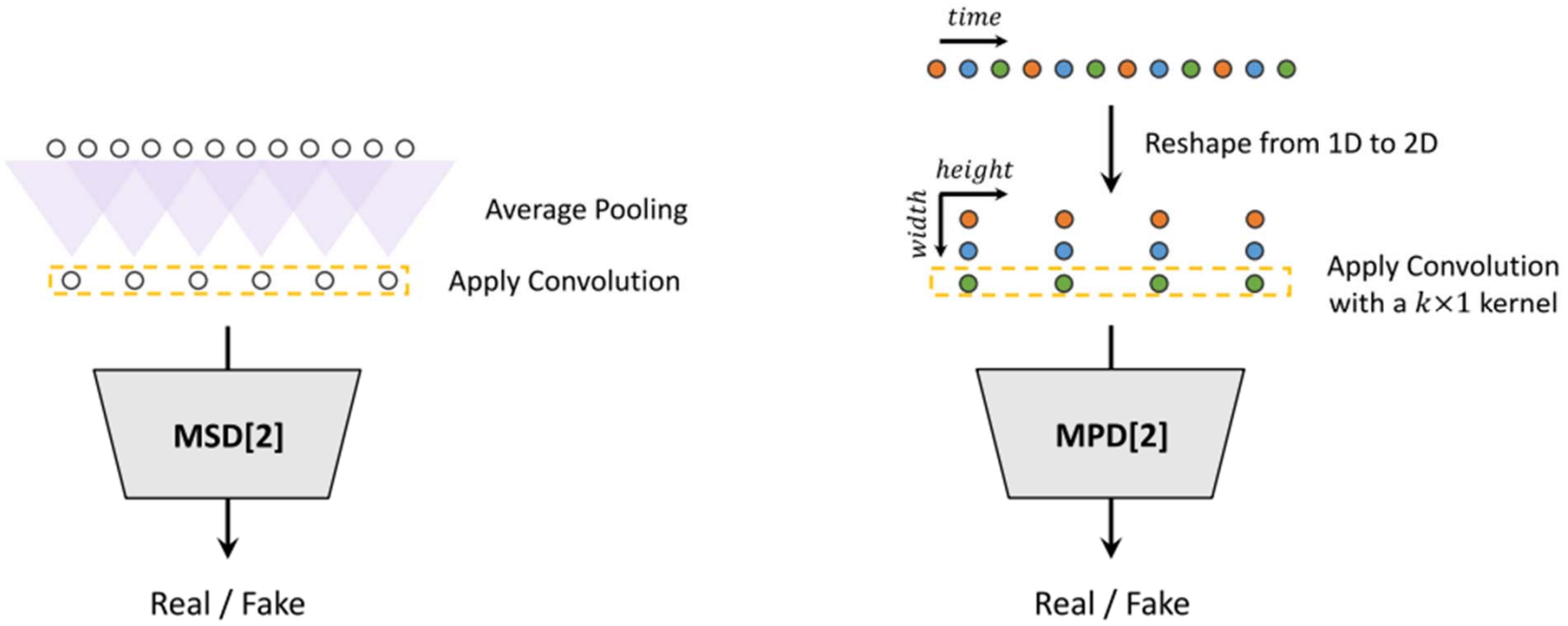


Table 5: Hyper-parameters of three generator $V1$, $V2$, and $V3$.

	Hyper-Parameters			
Model	h_u	k_u	k_r	D_r
$V1$	512	[16, 16, 4, 4]	[3, 7, 11]	[[1, 1], [3, 1], [5, 1]] ×3
$V2$	128	[16, 16, 4, 4]	[3, 7, 11]	[[1, 1], [3, 1], [5, 1]] ×3
$V3$	256	[16, 16, 8]	[3, 5, 7]	[[1], [2]], [[2], [6]], [[3], [12]]

HiFiGAN: Discriminator



- Multi-scale discriminator (MSD) + a new **multi-period discriminator** (MPD)
  - "MPD is a mixture of sub-discriminators, each of which only accepts equally spaced samples of an input audio [and therefore] looking at different parts of an input audio"
  - "We set the periods to [2, 3, 5, 7, 11] to avoid overlaps as much as possible."

HiFiGAN: Ablation Study

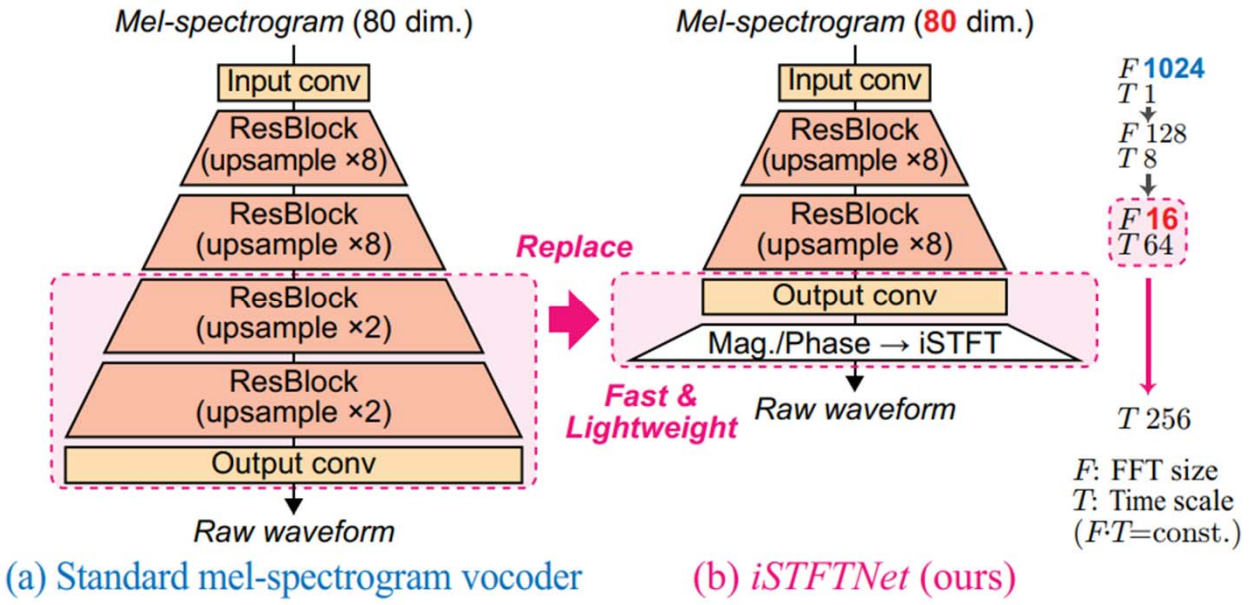
Model	MOS (CI)
Ground Truth	4.57 ($\pm$0.04)
Baseline (HiFi-GAN $V3$)	4.10 ($\pm$0.05)
w/o MPD	2.28 ($\pm$0.09)
w/o MSD	3.74 ($\pm$0.05)
w/o MRF	3.92 ($\pm$0.05)
w/o Mel-Spectrogram Loss	3.25 ($\pm$0.05)
MPD p=[2,4,8,16,32]	3.90 ($\pm$0.05)
MelGAN	2.88 ($\pm$0.08)
MelGAN with MPD	3.35 ($\pm$0.07)

Ref: Kong et al., “HiFi-GAN: Generative adversarial networks for efficient and high fidelity speech synthesis,” NeurIPS 2020

iSTFTNet

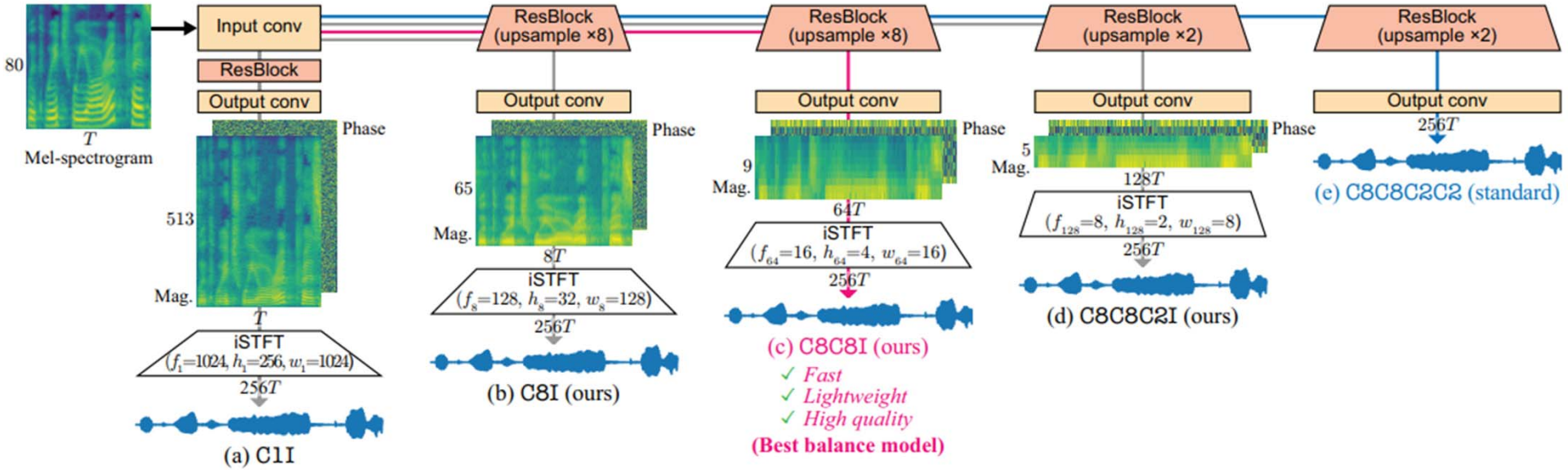
https://www.kecl.ntt.co.jp/people/kaneko.takuhiro/projects/istftnet/

- Improved version of HiFiGAN
- Use inverse STFT layers
  - replaces some layers after sufficiently reducing the frequency dimension using upsampling layers
  - reduces the computational cost from black-box modeling



Ref: Kaneko et al, "iSTFTNet: Fast and lightweight Mel-spectrogram vocoder incorporating Inverse Short-Time Fourier Transform," ICASSP 2022

iSTFTNet

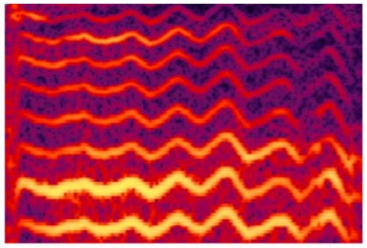


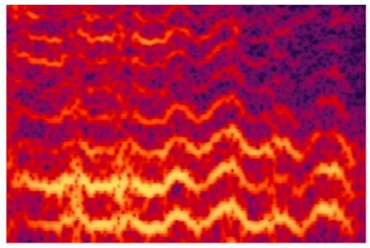
No.	Model	MOS↑	cFW2VD↓	Speed↑ (GPU)	Speed↑ (CPU)	# Param↓ (M)
1	Ground truth	4.46 ±0.14	–	–	–	–
2	V1 (original) [12]	4.22 ±0.17	0.020	×143.59 (100)	×1.34 (100)	13.94 (100)
3	V1-C8C8C2I	4.22 ±0.17	0.018	×179.42 (125)	×1.63 (122)	13.80 (99)
4	V1-C8C8I	4.26 ±0.17	0.020	×245.68 (171)	×2.33 (174)	13.26 (95)
5	V1-C8I	3.32 ±0.22	0.073	×609.43 (424)	×7.57 (565)	10.89 (78)
6	V1-C8C1I	3.82 ±0.17	0.033	×326.39 (227)	×3.97 (296)	19.15 (137)

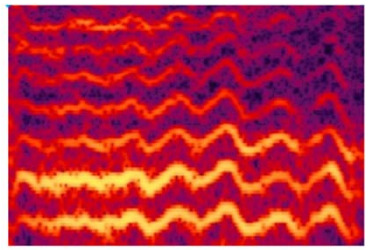
Ref: Kaneko et al, "iSTFTNet: Fast and lightweight Mel-spectrogram vocoder incorporating Inverse Short-Time Fourier Transform," ICASSP 2022

BigVGAN

- A **universal** vocoder that generalizes well for *zero-shot* (out-of-distribution) scenarios without fine-tuning
  - "We introduce **periodic activation function** and **anti-aliased representation** into the GAN generator, which brings the desired inductive bias for audio synthesis and significantly improves audio quality"
  - "We train our GAN vocoder at the largest scale up to **112M** parameters, which is unprecedented in the literature"
  - "BigVGAN trained only on **clean speech** (LibriTTS), achieves the SOTA performance for various zero-shot (out-of-distribution) conditions, including unseen speakers, languages, recording environments, **singing voices**, **music**, and **instrumental audio**"

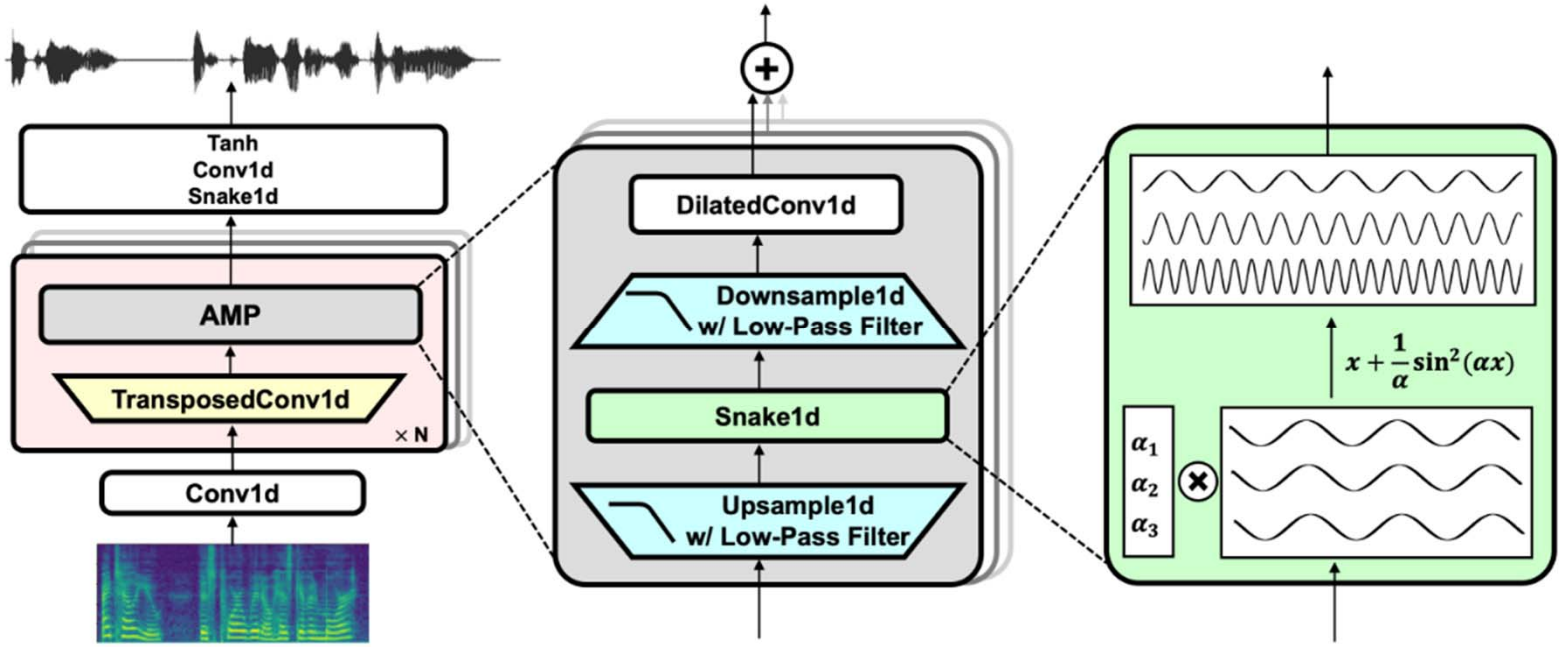

Ground Truth


HiFi-GAN


BigVGAN

Ref: Lee et al, "A universal neural vocoder with large-scale training," ICLR 2023

BigVGAN



- Anti-aliased multi-periodicity composition (AMP)
  - Uses **Snake** function for providing **periodic inductive bias**, and **low-pass filter** for **anti-aliasing** purpose.

Ref: Lee et al, "A universal neural vocoder with large-scale training," ICLR 2023